

AI Enabled VLSI Design, IIT Kharagpur

Assignment 1, 2, 3

Assignment 1 Problem Statement:

Functional simulation of MAC unit

- (a) Design a sequential MAC unit, with 8bit input data as well as weight coefficients being fed from input Ports sequentially. Assume input sequence length to be 8.
- (b) Design a sequential MAC unit with a local register file for storing weight coefficients. Assume a sequence length of 9. Both, inputs and weights are signed 8 bit numbers. Simulate for weight load and compute phases.
- (c) Design a fully parallel MAC unit with a local register file for storing weight coefficients. Assume a sequence length of 9. Both, inputs and weights are signed 8 bit numbers. Simulate for weight load and compute phases. Simulate for 2 cases : (i) without any pipelining inside the compute unit, (ii) with three-stage pipelining of the arithmetic unit. Verify the output waveforms in both cases.

Assignment 2 Problem Statement:

Simulate an SRAM of 64KB size. Simulate for two cases - (i) Assume one consolidated SRAM with read/write port of 32 bits (4 Bytes), Array size of 1024x 512. Simulate for write operation from test bench using for loop for any 8 locations. Verify the written data using test bench. (ii) Assume 4 banks of size 256 x 512. Verify bank wise read write operation for one location of each bank.

Assignment 3 Problem Statement:

Assume input SRAM module of 4 banks with 256 x 64 Bytes each, with 8bit read write port (all banks). Code and verify the address generation and read-control block for the first layer CNN operation, assuming 8 bit pixel values stored in a staggered manner, across 4 banks. Assume input image size of 256 x 256 and each pixel of 8 bit. The image is scanned row wise for each selection of column address. Assume 24-bit output bus for the SRAM.

Submitted by: KUSUMURU SURESH KUMAR
(kusumurusureshkumar21@gmail.com)

Find the codes at: https://github.com/suresh-vlsi/IITKGP_assignment

Assignment 1(a) — Sequential MAC, Fully Streamed Inputs:

1. mac_seq_a.sv:

```
`timescale 1ns/1ps

module mac_seq_a #(
    parameter W = 8,
    parameter SEQ_LEN = 8
) (
    input  logic clk,
    input  logic reset_n,
    input  logic valid_in,

    input  logic signed [W-1:0] data_in,
    input  logic signed [W-1:0] weight_in,

    output logic signed [2*W+3:0] mac_out,
    output logic out_valid
);

    // Accumulator
    logic signed [2*W+3:0] accumulator;

    // Counts valid input samples
    logic [3:0] count;

    // 8-bit signed × 8-bit signed = 16-bit signed product
    logic signed [2*W-1:0] product;

    assign product = data_in * weight_in;

    always_ff @(posedge clk) begin

        if (!reset_n) begin
            accumulator <= '0;
            count       <= '0;
            mac_out      <= '0;
            out_valid    <= 1'b0;
        end

        else begin

            // Default value
            out_valid <= 1'b0;

            if (valid_in) begin

                // Last sample of 8-sample sequence
                if (count == SEQ_LEN-1) begin

                    // Include the 8th multiplication
                    mac_out <= accumulator + product;

                    out_valid <= 1'b1;
                end
            end
        end
    end
endmodule
```

```

        // Prepare for next MAC sequence
        accumulator <= '0;
        count      <= '0;

    end

    else begin

        accumulator <= accumulator + product;
        count      <= count + 1'b1;

    end

end

end

end

endmodule

```

2. tb_mac_seq_a.sv:

```

`timescale 1ns/1ps

module tb_mac_seq_a;

    localparam W = 8;
    localparam SEQ_LEN = 8;

    logic clk;
    logic reset_n;
    logic valid_in;

    logic signed [W-1:0] data_in;
    logic signed [W-1:0] weight_in;

    logic signed [2*W+3:0] mac_out;
    logic out_valid;

    integer expected;

    // DUT
    mac_seq_a #(
        .W(W),
        .SEQ_LEN(SEQ_LEN)
    ) dut (
        .clk(clk),
        .reset_n(reset_n),
        .valid_in(valid_in),
        .data_in(data_in),
        .weight_in(weight_in),
        .mac_out(mac_out),
        .out_valid(out_valid)
    );

```

```

// 10 ns clock
initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

initial begin
    $dumpfile("mac_seq_a.vcd");
    $dumpvars(0, tb_mac_seq_a);

    reset_n    = 1'b0;
    valid_in   = 1'b0;
    data_in    = '0;
    weight_in  = '0;

    // Expected result:
    //  $10 \times 2 + 20 \times 3 + (-5) \times 4 + 7 \times (-6)$ 
    //  $+ 12 \times 5 + (-8) \times (-3) + 4 \times 9 + (-10) \times 2$ 
    expected = 118;

    // Reset
    repeat (2) @(posedge clk);

    reset_n = 1'b1;

    // -----
    // Input 1
    //  $10 \times 2 = 20$ 
    // -----
    @(negedge clk);
    valid_in = 1'b1;
    data_in  = 8'sd10;
    weight_in = 8'sd2;

    // -----
    // Input 2
    //  $20 \times 3 = 60$ 
    // -----
    @(negedge clk);
    data_in  = 8'sd20;
    weight_in = 8'sd3;

    // -----
    // Input 3
    //  $-5 \times 4 = -20$ 
    // -----
    @(negedge clk);
    data_in  = -8'sd5;
    weight_in = 8'sd4;

    // -----
    // Input 4
    //  $7 \times -6 = -42$ 
    // -----
    @(negedge clk);

```



```

data_in  = 8'sd7;
weight_in = -8'sd6;

// -----
// Input 5
//  $12 \times 5 = 60$ 
// -----
@(negedge clk);
data_in  = 8'sd12;
weight_in = 8'sd5;

// -----
// Input 6
//  $-8 \times -3 = 24$ 
// -----
@(negedge clk);
data_in  = -8'sd8;
weight_in = -8'sd3;

// -----
// Input 7
//  $4 \times 9 = 36$ 
// -----
@(negedge clk);
data_in  = 8'sd4;
weight_in = 8'sd9;

// -----
// Input 8
//  $-10 \times 2 = -20$ 
// -----
@(negedge clk);
data_in  = -8'sd10;
weight_in = 8'sd2;

// 8th MAC operation
@(posedge clk);

// Stop inputs
@(negedge clk);
valid_in  = 1'b0;
data_in  = '0;
weight_in = '0;

// Check result
@(posedge clk);

if (mac_out == expected) begin
    $display("");
    $display("=====");
    $display("          TEST PASSED");
    $display("=====");
    $display("Expected MAC = %0d", expected);
    $display("Actual MAC   = %0d", mac_out);
    $display("=====");
end

```

```

        $display("");
    end
    else begin
        $display("");
        $display("=====");
        $display("                TEST FAILED");
        $display("=====");
        $display("Expected MAC = %0d", expected);
        $display("Actual MAC   = %0d", mac_out);
        $display("=====");
        $display("");
    end

    #10;
    $finish;

end

// Debug information
always @(posedge clk) begin

    if (valid_in) begin
        $display(
            "TIME=%0t ns | DATA=%0d | WEIGHT=%0d | PRODUCT=%0d | COUNT=%0d |
ACC=%0d",
            $time,
            data_in,
            weight_in,
            data_in * weight_in,
            dut.count,
            dut.accumulator
        );
    end

    if (out_valid) begin
        $display(
            "TIME=%0t ns | MAC COMPLETE | MAC_OUT=%0d",
            $time,
            mac_out
        );
    end

end

endmodule

```

3. terminal:

The terminal window displays the output of a Verilog simulation for a MAC block. The simulation is run using the iverilog tool, and the results show the MAC value calculated for different input data and weights. The simulation is successful, and the expected MAC value is 118.

```
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/1a$ iverilog -g2012 -o mac_seq_a_sim mac_seq_a.vv tb_mac_seq_a.vv
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/1a$ vvp mac_seq_a_sim
VCD info: dumpfile mac_seq_a.vcd opened for output.
TIME=25000 ns | DATA=10 | WEIGHT=2 | PRODUCT=20 | COUNT=0 | ACC=0
TIME=35000 ns | DATA=20 | WEIGHT=3 | PRODUCT=60 | COUNT=1 | ACC=20
TIME=45000 ns | DATA=-5 | WEIGHT=4 | PRODUCT=-20 | COUNT=2 | ACC=80
TIME=55000 ns | DATA=7 | WEIGHT=6 | PRODUCT=42 | COUNT=3 | ACC=60
TIME=65000 ns | DATA=12 | WEIGHT=5 | PRODUCT=60 | COUNT=4 | ACC=18
TIME=75000 ns | DATA=-8 | WEIGHT=3 | PRODUCT=-24 | COUNT=5 | ACC=78
TIME=85000 ns | DATA=4 | WEIGHT=9 | PRODUCT=36 | COUNT=6 | ACC=102
TIME=95000 ns | DATA=-10 | WEIGHT=2 | PRODUCT=-20 | COUNT=7 | ACC=138

=====
TEST PASSED
=====
Expected MAC = 118
Actual MAC = 118
=====

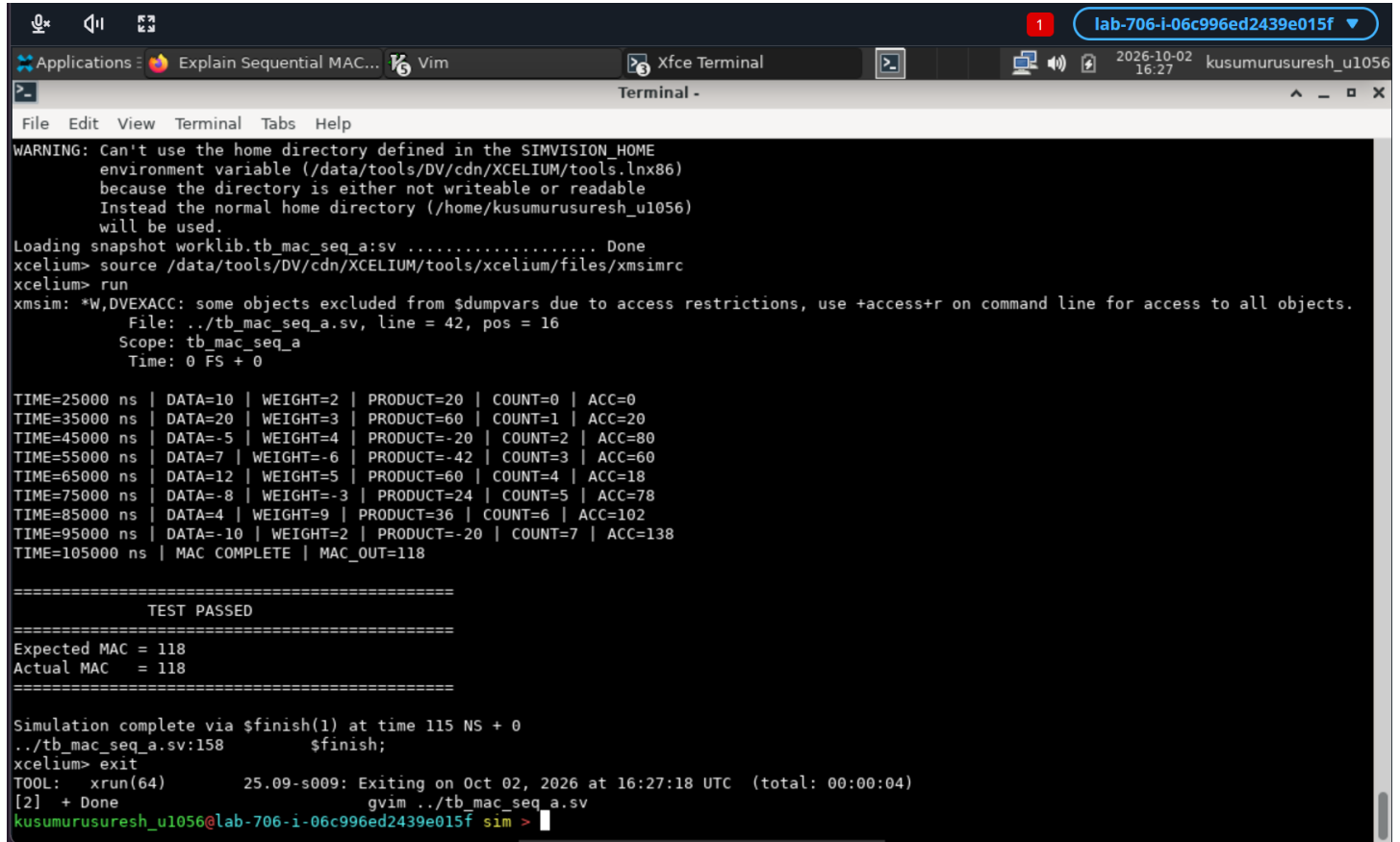
TIME=105000 ns | MAC COMPLETE | MAC_OUT=118
tb_mac_seq_a.vv:158: $finish called at 115000 (1ps)
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/1a$ ls -lh
total 28K
-rw-r--r-- 1 suresh@lsid suresh@lsid 1.5K Sep 26 04:49 mac_seq_a.vv
-rw-r--r-- 1 suresh@lsid suresh@lsid 1.9K Sep 26 05:01 mac_seq_a.vcd
-rwxr-xr-x 1 suresh@lsid suresh@lsid 8.8K Sep 26 05:01 mac_seq_a_sim
-rw-r--r-- 1 suresh@lsid suresh@lsid 4.8K Sep 26 04:51 tb_mac_seq_a.vv
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/1a$ gtwave mac_seq_a.vcd &
[1] 11131
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/1a$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
o suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/1a$
```

4. gtwave:

The GTKWave waveform viewer displays the simulation results for the MAC block. The signals are plotted over time, showing the data input, weight input, product, count, and accumulator. The waveform shows the data input, weight input, product, count, and accumulator signals over time. The data input is a 7-bit signal, the weight input is a 7-bit signal, the product is a 15-bit signal, the count is a 3-bit signal, and the accumulator is a 19-bit signal. The waveform shows the data input, weight input, product, count, and accumulator signals over time. The data input is a 7-bit signal, the weight input is a 7-bit signal, the product is a 15-bit signal, the count is a 3-bit signal, and the accumulator is a 19-bit signal. The waveform shows the data input, weight input, product, count, and accumulator signals over time.

5. Cadence Xcelium simulation environment terminal:



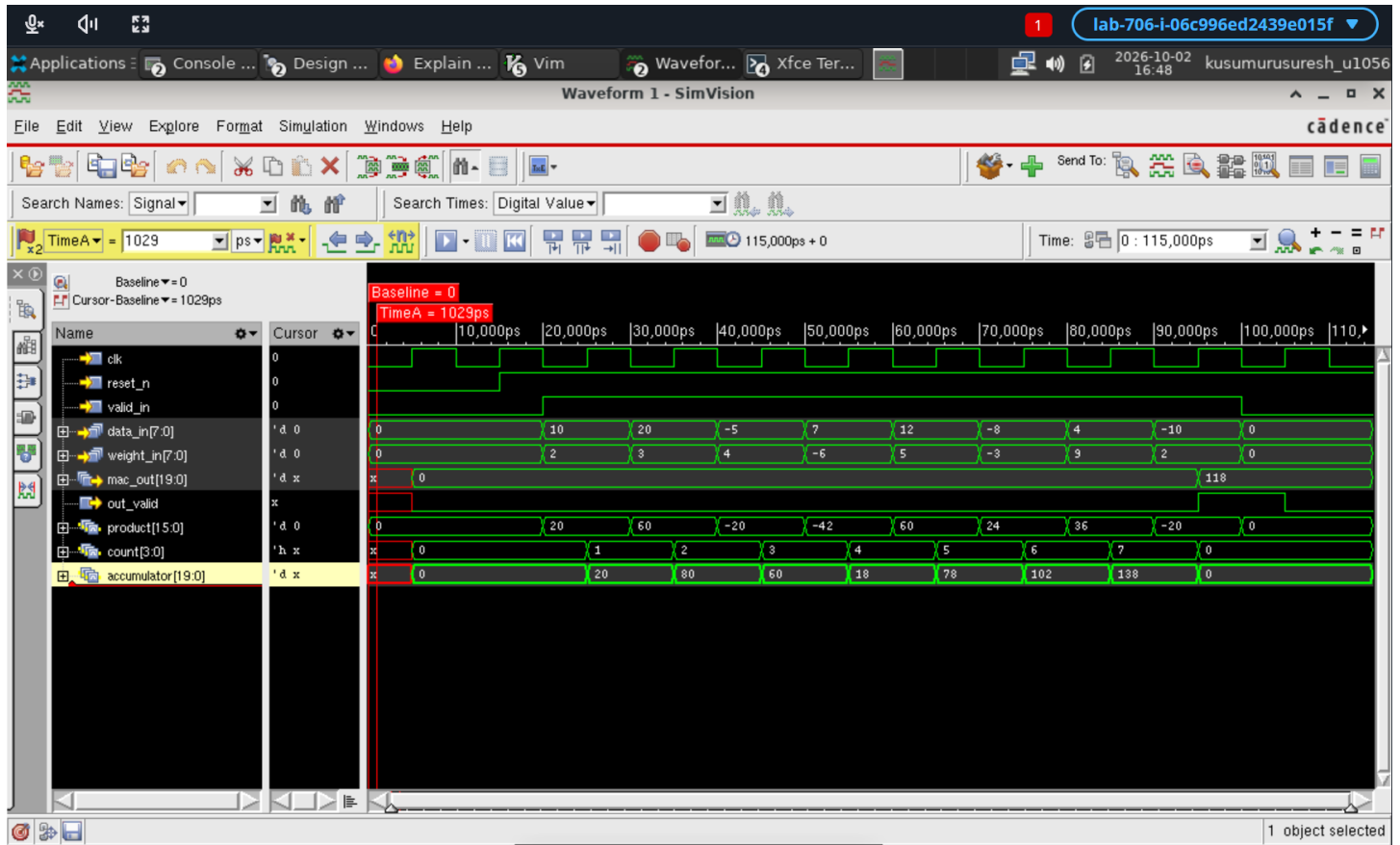
```
WARNING: Can't use the home directory defined in the SIMVISION_HOME
environment variable (/data/tools/DV/cdn/XCELIUM/tools.lnx86)
because the directory is either not writeable or readable
Instead the normal home directory (/home/kusumurusresh_u1056)
will be used.
Loading snapshot worklib.tb_mac_seq_a:sv ..... Done
xcelium> source /data/tools/DV/cdn/XCELIUM/tools/xcelium/files/xmsimrc
xcelium> run
xmsim: *W,DVEXACC: some objects excluded from $dumpvars due to access restrictions, use +access+r on command line for access to all objects.
File: ../tb_mac_seq_a.sv, line = 42, pos = 16
Scope: tb_mac_seq_a
Time: 0 FS + 0

TIME=25000 ns | DATA=10 | WEIGHT=2 | PRODUCT=20 | COUNT=0 | ACC=0
TIME=35000 ns | DATA=20 | WEIGHT=3 | PRODUCT=60 | COUNT=1 | ACC=20
TIME=45000 ns | DATA=-5 | WEIGHT=4 | PRODUCT=-20 | COUNT=2 | ACC=80
TIME=55000 ns | DATA=7 | WEIGHT=-6 | PRODUCT=-42 | COUNT=3 | ACC=60
TIME=65000 ns | DATA=12 | WEIGHT=5 | PRODUCT=60 | COUNT=4 | ACC=18
TIME=75000 ns | DATA=-8 | WEIGHT=-3 | PRODUCT=24 | COUNT=5 | ACC=78
TIME=85000 ns | DATA=4 | WEIGHT=9 | PRODUCT=36 | COUNT=6 | ACC=102
TIME=95000 ns | DATA=-10 | WEIGHT=2 | PRODUCT=-20 | COUNT=7 | ACC=138
TIME=105000 ns | MAC COMPLETE | MAC_OUT=118

=====
TEST PASSED
=====
Expected MAC = 118
Actual MAC = 118
=====

Simulation complete via $finish(1) at time 115 NS + 0
../tb_mac_seq_a.sv:158 $finish;
xcelium> exit
T00L: xrun(64) 25.09-s009: Exiting on Oct 02, 2026 at 16:27:18 UTC (total: 00:00:04)
[2] + Done gvim ../tb_mac_seq_a.sv
kusumurusresh_u1056@lab-706-i-06c996ed2439e015f sim >
```

6. Cadence SimVision Waveform Viewer:





Assignment 1(b) — Sequential MAC with Local Register File:

1. mac_seq_b.sv:

```
`timescale 1ns/1ps

module mac_seq_b #(
    parameter W = 8,
    parameter N = 9
)(
    input  logic clk,
    input  logic reset_n,

    // Weight loading phase
    input  logic load_weight,
    input  logic signed [W-1:0] weight_in,

    // Compute phase
    input  logic signed [W-1:0] data_in,

    output logic signed [2*W+4:0] mac_out,
    output logic out_valid
);

    // -----
    // Local weight register file
    // -----
    logic signed [W-1:0] kernel_buff [0:N-1];

    // -----
    // Load and compute counters
    // -----
    logic [3:0] load_idx;
    logic [3:0] comp_idx;

    // -----
    // Current weight and multiplication
    // -----
    logic signed [W-1:0] current_weight;
    logic signed [2*W-1:0] product;

    // -----
    // Accumulator
    // -----
    logic signed [2*W+4:0] accumulator;

    // Weight selected during compute
    assign current_weight = kernel_buff[comp_idx];

    // Signed multiplication
    assign product = data_in * current_weight;

    // -----
    // Sequential logic
    // -----
    always_ff @(posedge clk) begin
```

```

if (!reset_n) begin

    load_idx    <= '0;
    comp_idx    <= '0;

    accumulator <= '0;
    mac_out     <= '0;

    out_valid <= 1'b0;

end

else begin

    // Default
    out_valid <= 1'b0;

    // -----
    // WEIGHT LOAD PHASE
    // -----
    if (load_weight) begin

        kernel_buff[load_idx] <= weight_in;

        if (load_idx == N-1) begin
            load_idx <= '0;
            comp_idx <= '0;
            accumulator <= '0;
        end
        else begin
            load_idx <= load_idx + 1'b1;
        end

    end

    // -----
    // COMPUTE PHASE
    // -----
    else begin

        if (comp_idx == N-1) begin

            // Include final product
            mac_out <= accumulator + product;

            out_valid <= 1'b1;

            // Prepare for next sequence
            accumulator <= '0;
            comp_idx <= '0;

        end

    end

end

```

```

        accumulator <= accumulator + product;

        comp_idx <= comp_idx + 1'b1;

    end

end

end

end

endmodule

```

2. tb mac seq b:

```

`timescale 1ns/1ps

module tb_mac_seq_b;

    localparam W = 8;
    localparam N = 9;

    logic clk;
    logic reset_n;

    logic load_weight;

    logic signed [W-1:0] weight_in;
    logic signed [W-1:0] data_in;

    logic signed [2*W+4:0] mac_out;
    logic out_valid;

    integer expected;

    // =====
    // DUT
    // =====

    mac_seq_b #(
        .W(W),
        .N(N)
    ) dut (
        .clk(clk),
        .reset_n(reset_n),
        .load_weight(load_weight),
        .weight_in(weight_in),
        .data_in(data_in),
        .mac_out(mac_out),
        .out_valid(out_valid)
    );

    // =====

```



```

// CLOCK
// =====

initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

// =====
// TEST
// =====

initial begin

    // -----
    // Waveform
    // -----

    $dumpfile("mac_seq_b.vcd");
    $dumpvars(0, tb_mac_seq_b);

    // -----
    // Initial values
    // -----

    reset_n    = 1'b0;
    load_weight = 1'b0;
    weight_in   = '0;
    data_in     = '0;

    // Expected result:
    //
    // 10 * 2 = 20
    // 20 * 3 = 60
    // -5 * 4 = -20
    // 7 * -6 = -42
    // 12 * 5 = 60
    // -8 * -3 = 24
    // 4 * 9 = 36
    // -10 * 2 = -20
    // 6 * -4 = -24
    //
    // TOTAL = 94

    expected = 94;

    // =====
    // RESET
    // =====

    repeat (2) @(posedge clk);

    reset_n = 1'b1;

    // =====

```

```

// WEIGHT LOAD PHASE
// =====

// Weight 0 = +2
@(negedge clk);
load_weight = 1'b1;
weight_in = 8'sd2;

// Weight 1 = +3
@(negedge clk);
weight_in = 8'sd3;

// Weight 2 = +4
@(negedge clk);
weight_in = 8'sd4;

// Weight 3 = -6
@(negedge clk);
weight_in = -8'sd6;

// Weight 4 = +5
@(negedge clk);
weight_in = 8'sd5;

// Weight 5 = -3
@(negedge clk);
weight_in = -8'sd3;

// Weight 6 = +9
@(negedge clk);
weight_in = 8'sd9;

// Weight 7 = +2
@(negedge clk);
weight_in = 8'sd2;

// Weight 8 = -4
@(negedge clk);
weight_in = -8'sd4;

// =====
// TRANSITION TO COMPUTE
// =====
//
// IMPORTANT:
// Put the FIRST DATA VALUE here BEFORE the next
// positive clock edge.
//
// This prevents an unwanted data=0 compute cycle.
// =====

@(negedge clk);
load_weight = 1'b0;
weight_in   = '0;
data_in     = 8'sd10;

```

```

// =====
// COMPUTE PHASE
// =====

// Data 0 = +10
@(posedge clk);

// Data 1 = +20
@(negedge clk);
data_in = 8'sd20;

// Data 2 = -5
@(negedge clk);
data_in = -8'sd5;

// Data 3 = +7
@(negedge clk);
data_in = 8'sd7;

// Data 4 = +12
@(negedge clk);
data_in = 8'sd12;

// Data 5 = -8
@(negedge clk);
data_in = -8'sd8;

// Data 6 = +4
@(negedge clk);
data_in = 8'sd4;

// Data 7 = -10
@(negedge clk);
data_in = -8'sd10;

// Data 8 = +6
@(negedge clk);
data_in = 8'sd6;

// =====
// 9th COMPUTE OPERATION
// =====

@(posedge clk);

// Allow nonblocking assignments to update
#1;

// =====
// CHECK RESULT IMMEDIATELY
// =====

if (out_valid && (mac_out == expected)) begin

```

```

        $display("");
        $display("=====");
        $display("          ASSIGNMENT 1(b) TEST PASSED");
        $display("=====");
        $display("Expected MAC = %0d", expected);
        $display("Actual MAC   = %0d", mac_out);
        $display("out_valid   = %b", out_valid);
        $display("=====");
        $display("");

    end
    else begin

        $display("");
        $display("=====");
        $display("          ASSIGNMENT 1(b) TEST FAILED");
        $display("=====");
        $display("Expected MAC = %0d", expected);
        $display("Actual MAC   = %0d", mac_out);
        $display("out_valid   = %b", out_valid);
        $display("=====");
        $display("");

    end

    // Stop simulation
    #10;
    $finish;

end

// =====
// DEBUG OUTPUT
// =====

always @(posedge clk) begin

    if (!reset_n) begin

        $display(
            "RESET | TIME=%0t ns",
            $time
        );

    end

    else if (load_weight) begin

        $display(
            "LOAD | TIME=%0t ns | index=%0d | weight=%0d",
            $time,
            dut.load_idx,
            weight_in
        );

    end
end

```

```
end

else begin

    $display(
        "COMPUTE | TIME=%0t ns | index=%0d | data=%0d | weight=%0d |
product=%0d | acc=%0d",
        $time,
        dut.comp_idx,
        data_in,
        dut.current_weight,
        dut.product,
        dut.accumulator
    );

end

if (out_valid) begin

    $display(
        "RESULT | TIME=%0t ns | MAC_OUT=%0d",
        $time,
        mac_out
    );

end

end

endmodule
```

3. terminal:

The screenshot shows a VS Code terminal window with the following content:

```
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1b$ iverilog -g2012 -o mac_seq_b_sim mac_seq_b.sv tb_mac_seq_b.sv
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1b$ vvp mac_seq_b_sim
VCD info: dumpfile mac_seq_b.vcd opened for output.
RESET | TIME=5000 ns
COMPUTE | TIME=15000 ns | index=0 | data=0 | weight=x | product=x | acc=0
LOAD | TIME=25000 ns | index=0 | weight=2
LOAD | TIME=35000 ns | index=1 | weight=3
LOAD | TIME=45000 ns | index=2 | weight=4
LOAD | TIME=55000 ns | index=3 | weight=6
LOAD | TIME=65000 ns | index=4 | weight=5
LOAD | TIME=75000 ns | index=5 | weight=3
LOAD | TIME=85000 ns | index=6 | weight=9
LOAD | TIME=95000 ns | index=7 | weight=2
LOAD | TIME=105000 ns | index=8 | weight=-4
COMPUTE | TIME=115000 ns | index=0 | data=10 | weight=2 | product=20 | acc=0
COMPUTE | TIME=125000 ns | index=1 | data=20 | weight=3 | product=60 | acc=20
COMPUTE | TIME=135000 ns | index=2 | data=5 | weight=4 | product=-20 | acc=80
COMPUTE | TIME=145000 ns | index=3 | data=7 | weight=6 | product=-42 | acc=60
COMPUTE | TIME=155000 ns | index=4 | data=12 | weight=5 | product=60 | acc=18
COMPUTE | TIME=165000 ns | index=5 | data=8 | weight=3 | product=24 | acc=78
COMPUTE | TIME=175000 ns | index=6 | data=4 | weight=9 | product=36 | acc=102
COMPUTE | TIME=185000 ns | index=7 | data=10 | weight=2 | product=-20 | acc=138
COMPUTE | TIME=195000 ns | index=8 | data=6 | weight=-4 | product=-24 | acc=118

=====
ASSIGNMENT 1(b) TEST PASSED
=====
Expected MAC = 94
Actual MAC = 94
out_valid = 1
=====

COMPUTE | TIME=205000 ns | index=0 | data=6 | weight=2 | product=12 | acc=0
RESULT | TIME=205000 ns | MAC_OUT=94
tb_mac_seq_b.sv:231: $finish called at 206000 (1ps)
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1b$ gtkwave mac_seq_b.vcd &
[1] 19557
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1b$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1b$
```

The terminal output shows the execution of the Verilog simulation, including the loading of weights, computation of the MAC, and the final test results. The test passed, and the expected MAC (94) matches the actual MAC (94).

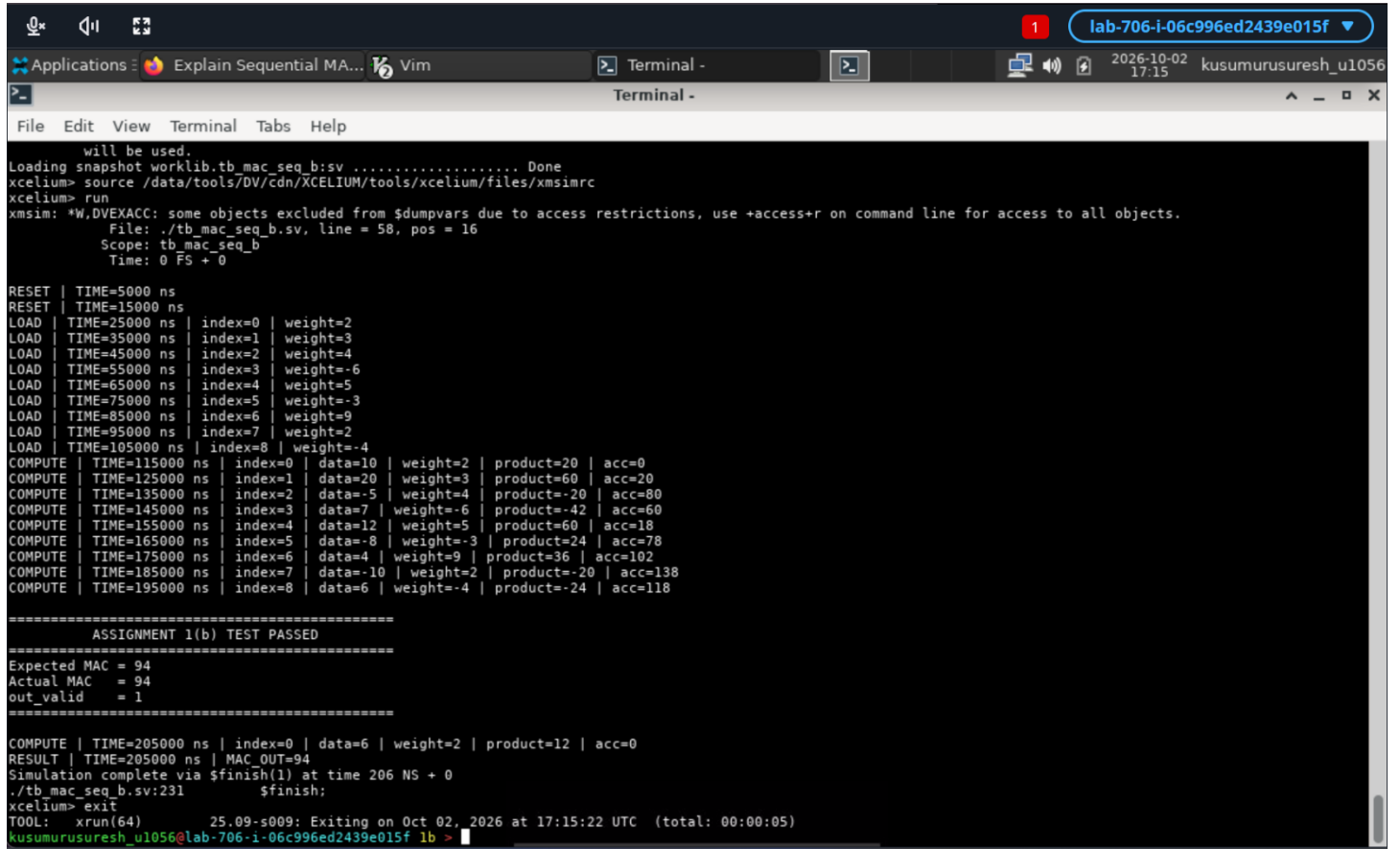
4. gtkwave:

The screenshot shows the GTKWave interface with the following components:

- Signals List:** A list of signals including `tb_mac_seq_b`, `accutator[20:0]`, `clk`, `comp_idx[3:0]`, `current_weight[7:0]`, `data_in[7:0]`, `load_idx[3:0]`, `load_weight`, `mac_out[20:0]`, `out_valid`, `product[15:0]`, `reset_n`, and `weight_in[7:0]`.
- Waveform View:** A time-based waveform view showing the signals over time. The time scale ranges from 0 ns to 206 ns. The signals are plotted as digital waveforms.
- Search and Edit:** A search bar and buttons for Append, Insert, and Replace.

The waveform view shows the signals over time, with the time scale ranging from 0 ns to 206 ns. The signals are plotted as digital waveforms, showing the sequence of events during the simulation.

5. Cadence Xcelium simulation environment terminal:



The screenshot shows the Cadence Xcelium simulation environment terminal window. The terminal displays the following output:

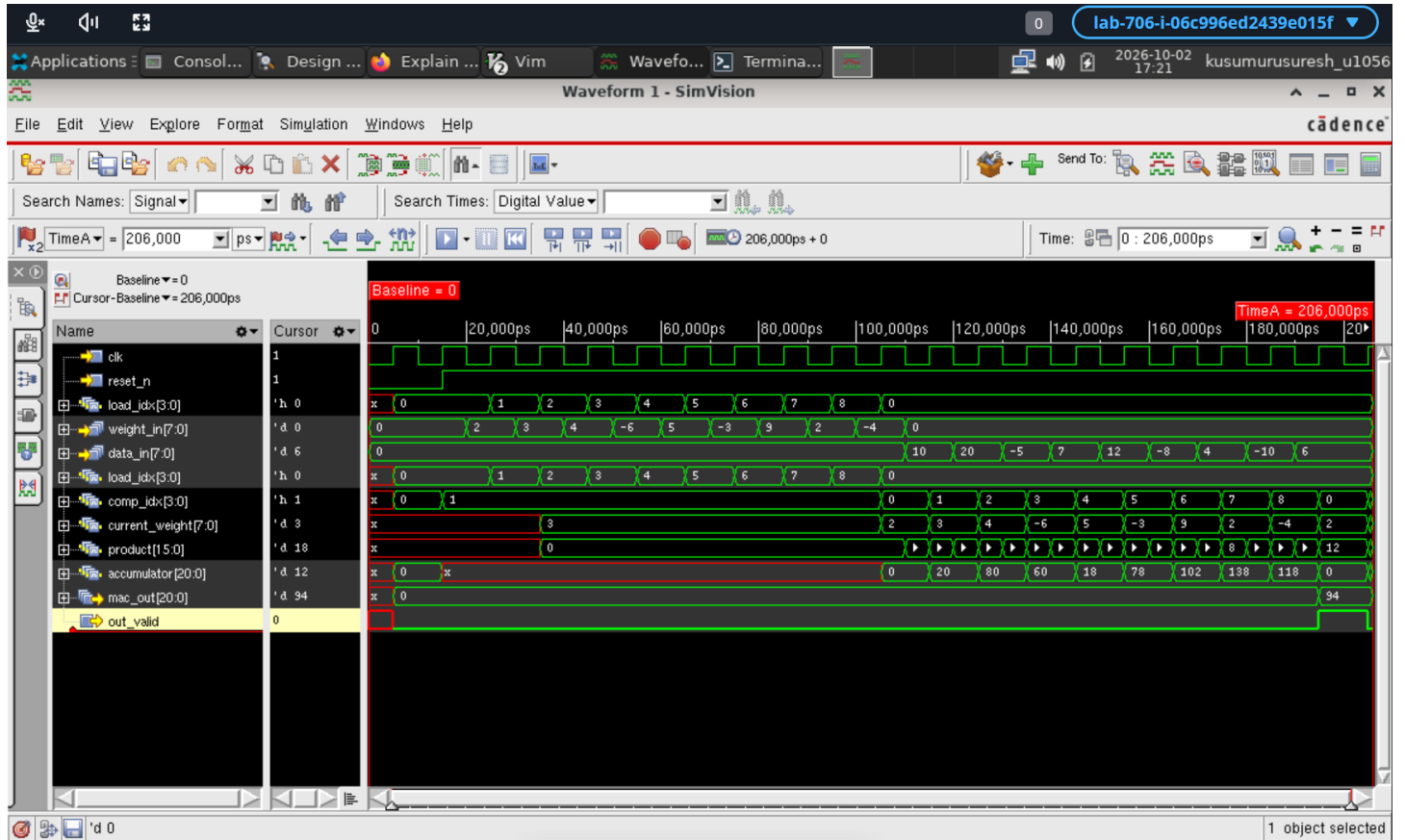
```
will be used.
Loading snapshot worklib.tb_mac_seq_b.sv ..... Done
xcelium> source /data/tools/DV/Cdn/XCELIUM/tools/xcelium/files/xmsimrc
xcelium> run
xmsim: *W,DVEXACC: some objects excluded from $dumpvars due to access restrictions, use +access+r on command line for access to all objects.
File: ./tb_mac_seq_b.sv, line = 58, pos = 16
Scope: tb_mac_seq_b
Time: 0 Fs +0

RESET | TIME=5000 ns
RESET | TIME=15000 ns
LOAD | TIME=25000 ns | index=0 | weight=2
LOAD | TIME=35000 ns | index=1 | weight=3
LOAD | TIME=45000 ns | index=2 | weight=4
LOAD | TIME=55000 ns | index=3 | weight=-6
LOAD | TIME=65000 ns | index=4 | weight=5
LOAD | TIME=75000 ns | index=5 | weight=-3
LOAD | TIME=85000 ns | index=6 | weight=9
LOAD | TIME=95000 ns | index=7 | weight=2
LOAD | TIME=105000 ns | index=8 | weight=-4
COMPUTE | TIME=115000 ns | index=0 | data=10 | weight=2 | product=20 | acc=0
COMPUTE | TIME=125000 ns | index=1 | data=20 | weight=3 | product=60 | acc=20
COMPUTE | TIME=135000 ns | index=2 | data=-5 | weight=4 | product=-20 | acc=80
COMPUTE | TIME=145000 ns | index=3 | data=7 | weight=6 | product=42 | acc=60
COMPUTE | TIME=155000 ns | index=4 | data=12 | weight=5 | product=60 | acc=18
COMPUTE | TIME=165000 ns | index=5 | data=-8 | weight=3 | product=24 | acc=78
COMPUTE | TIME=175000 ns | index=6 | data=4 | weight=9 | product=36 | acc=102
COMPUTE | TIME=185000 ns | index=7 | data=-10 | weight=2 | product=-20 | acc=138
COMPUTE | TIME=195000 ns | index=8 | data=6 | weight=-4 | product=-24 | acc=118

=====
ASSIGNMENT 1(b) TEST PASSED
=====
Expected MAC = 94
Actual MAC = 94
out_valid = 1

=====
COMPUTE | TIME=205000 ns | index=0 | data=6 | weight=2 | product=12 | acc=0
RESULT | TIME=205000 ns | MAC_OUT=94
Simulation complete via $finish(l) at time 206 NS + 0
./tb_mac_seq_b.sv:231 $finish:
xcelium> exit
TOOL: xrun(64) 25.09-s009: Exiting on Oct 02, 2026 at 17:15:22 UTC (total: 00:00:05)
ksumurusuresh_u1056@lab-706-i-06c996ed2439e015f lb >
```

6. Cadence SimVision Waveform Viewer:





Assignment 1(c) — Parallel MAC, Pipelined vs Non-Pipelined:

Non-Pipelined:

1. mac_parallel.sv

```
`timescale 1ns/1ps

module mac_parallel #(
    parameter W = 8,
    parameter N = 9
) (
    input  logic clk,
    input  logic reset_n,
    input  logic valid_in,

    input  logic signed [W-1:0] data0,
    input  logic signed [W-1:0] data1,
    input  logic signed [W-1:0] data2,
    input  logic signed [W-1:0] data3,
    input  logic signed [W-1:0] data4,
    input  logic signed [W-1:0] data5,
    input  logic signed [W-1:0] data6,
    input  logic signed [W-1:0] data7,
    input  logic signed [W-1:0] data8,

    input  logic signed [W-1:0] weight0,
    input  logic signed [W-1:0] weight1,
    input  logic signed [W-1:0] weight2,
    input  logic signed [W-1:0] weight3,
    input  logic signed [W-1:0] weight4,
    input  logic signed [W-1:0] weight5,
    input  logic signed [W-1:0] weight6,
    input  logic signed [W-1:0] weight7,
    input  logic signed [W-1:0] weight8,

    output logic signed [2*W+4:0] mac_out,
    output logic out_valid
);

// =====
// Nine parallel multipliers
// =====

logic signed [2*W-1:0] product0;
logic signed [2*W-1:0] product1;
logic signed [2*W-1:0] product2;
logic signed [2*W-1:0] product3;
logic signed [2*W-1:0] product4;
logic signed [2*W-1:0] product5;
logic signed [2*W-1:0] product6;
logic signed [2*W-1:0] product7;
logic signed [2*W-1:0] product8;

assign product0 = data0 * weight0;
assign product1 = data1 * weight1;
assign product2 = data2 * weight2;
```

```

assign product3 = data3 * weight3;
assign product4 = data4 * weight4;
assign product5 = data5 * weight5;
assign product6 = data6 * weight6;
assign product7 = data7 * weight7;
assign product8 = data8 * weight8;

// =====
// Adder tree
// =====

logic signed [2*W:0] sum01;
logic signed [2*W:0] sum23;
logic signed [2*W:0] sum45;
logic signed [2*W:0] sum67;

logic signed [2*W+1:0] sum0123;
logic signed [2*W+1:0] sum4567;

logic signed [2*W+2:0] sum_tree;

assign sum01 = product0 + product1;
assign sum23 = product2 + product3;
assign sum45 = product4 + product5;
assign sum67 = product6 + product7;

assign sum0123 = sum01 + sum23;
assign sum4567 = sum45 + sum67;

assign sum_tree = sum0123 + sum4567 + product8;

// =====
// Output
// =====

always_comb begin

    if (!reset_n) begin
        mac_out  = '0;
        out_valid = 1'b0;
    end

    else begin
        mac_out  = sum_tree;
        out_valid = valid_in;
    end

end

endmodule

```

2.tb_mac_parallel.sv

```

`timescale 1ns/1ps

module tb_mac_parallel;

```

```

logic clk;
logic reset_n;
logic valid_in;

logic signed [7:0] data0;
logic signed [7:0] data1;
logic signed [7:0] data2;
logic signed [7:0] data3;
logic signed [7:0] data4;
logic signed [7:0] data5;
logic signed [7:0] data6;
logic signed [7:0] data7;
logic signed [7:0] data8;

logic signed [7:0] weight0;
logic signed [7:0] weight1;
logic signed [7:0] weight2;
logic signed [7:0] weight3;
logic signed [7:0] weight4;
logic signed [7:0] weight5;
logic signed [7:0] weight6;
logic signed [7:0] weight7;
logic signed [7:0] weight8;

logic signed [20:0] mac_out;
logic out_valid;

integer expected;

// =====
// DUT
// =====

mac_parallel dut (
    .clk(clk),
    .reset_n(reset_n),
    .valid_in(valid_in),

    .data0(data0),
    .data1(data1),
    .data2(data2),
    .data3(data3),
    .data4(data4),
    .data5(data5),
    .data6(data6),
    .data7(data7),
    .data8(data8),

    .weight0(weight0),
    .weight1(weight1),
    .weight2(weight2),
    .weight3(weight3),
    .weight4(weight4),
    .weight5(weight5),

```

```

        .weight6(weight6),
        .weight7(weight7),
        .weight8(weight8),

        .mac_out(mac_out),
        .out_valid(out_valid)
    );

// =====
// CLOCK
// =====

initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

// =====
// TEST
// =====

initial begin

    $dumpfile("mac_parallel.vcd");
    $dumpvars(0, tb_mac_parallel);

    reset_n = 1'b0;
    valid_in = 1'b0;

    data0 = 0;
    data1 = 0;
    data2 = 0;
    data3 = 0;
    data4 = 0;
    data5 = 0;
    data6 = 0;
    data7 = 0;
    data8 = 0;

    weight0 = 0;
    weight1 = 0;
    weight2 = 0;
    weight3 = 0;
    weight4 = 0;
    weight5 = 0;
    weight6 = 0;
    weight7 = 0;
    weight8 = 0;

    expected = 94;

    // =====
    // RESET
    // =====

```

```

#12;
reset_n = 1'b1;

// =====
// PRESENT ALL 9 DATA/WEIGHT PAIRS SIMULTANEOUSLY
// =====

@(negedge clk);

data0 = 8'sd10;
data1 = 8'sd20;
data2 = -8'sd5;
data3 = 8'sd7;
data4 = 8'sd12;
data5 = -8'sd8;
data6 = 8'sd4;
data7 = -8'sd10;
data8 = 8'sd6;

weight0 = 8'sd2;
weight1 = 8'sd3;
weight2 = 8'sd4;
weight3 = -8'sd6;
weight4 = 8'sd5;
weight5 = -8'sd3;
weight6 = 8'sd9;
weight7 = 8'sd2;
weight8 = -8'sd4;

valid_in = 1'b1;

// =====
// CHECK
// =====

#1;

if (out_valid && (mac_out == expected)) begin

    $display("");
    $display("=====");
    $display("      1(c)(i) NON-PIPELINED TEST PASSED");
    $display("=====");
    $display("Expected MAC = %0d", expected);
    $display("Actual MAC   = %0d", mac_out);
    $display("out_valid    = %b", out_valid);
    $display("=====");
    $display("");

end
else begin

    $display("");
    $display("=====");
    $display("      1(c)(i) NON-PIPELINED TEST FAILED");

```

```

        $display("=====");
        $display("Expected MAC = %0d", expected);
        $display("Actual MAC    = %0d", mac_out);
        $display("out_valid    = %b", out_valid);
        $display("=====");
        $display("");

    end

    valid_in = 1'b0;

    #20;
    $finish;

end

endmodule

```

3. terminal:

The screenshot shows a VS Code terminal window with the following content:

```

sureshvlisi@DESKTOP-CB90UTN:~/IITKGP_assignment1/lc/non_pipelined$ iverilog -g2012 -o mac_parallel_sim mac_parallel.sv
v tb_mac_parallel.sv
sureshvlisi@DESKTOP-CB90UTN:~/IITKGP_assignment1/lc/non_pipelined$ vvp mac_parallel_sim
VCD info: dumpfile mac_parallel.vcd opened for output.

=====
1(c)(i) NON-PIPELINED TEST PASSED
=====
Expected MAC = 94
Actual MAC    = 94
out_valid     = 1
=====

tb_mac_parallel.sv:181: $finish called at 41000 (1ps)
sureshvlisi@DESKTOP-CB90UTN:~/IITKGP_assignment1/lc/non_pipelined$
sureshvlisi@DESKTOP-CB90UTN:~/IITKGP_assignment1/lc/non_pipelined$ gtkwave mac_parallel.vcd &
[1] 24648
sureshvlisi@DESKTOP-CB90UTN:~/IITKGP_assignment1/lc/non_pipelined$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[41000] end time.

```

The terminal output shows the successful execution of the Verilog code, with the test passing and the expected MAC value of 94 matching the actual MAC value of 94. The simulation ends at 41000 ps.

[illegible]

```

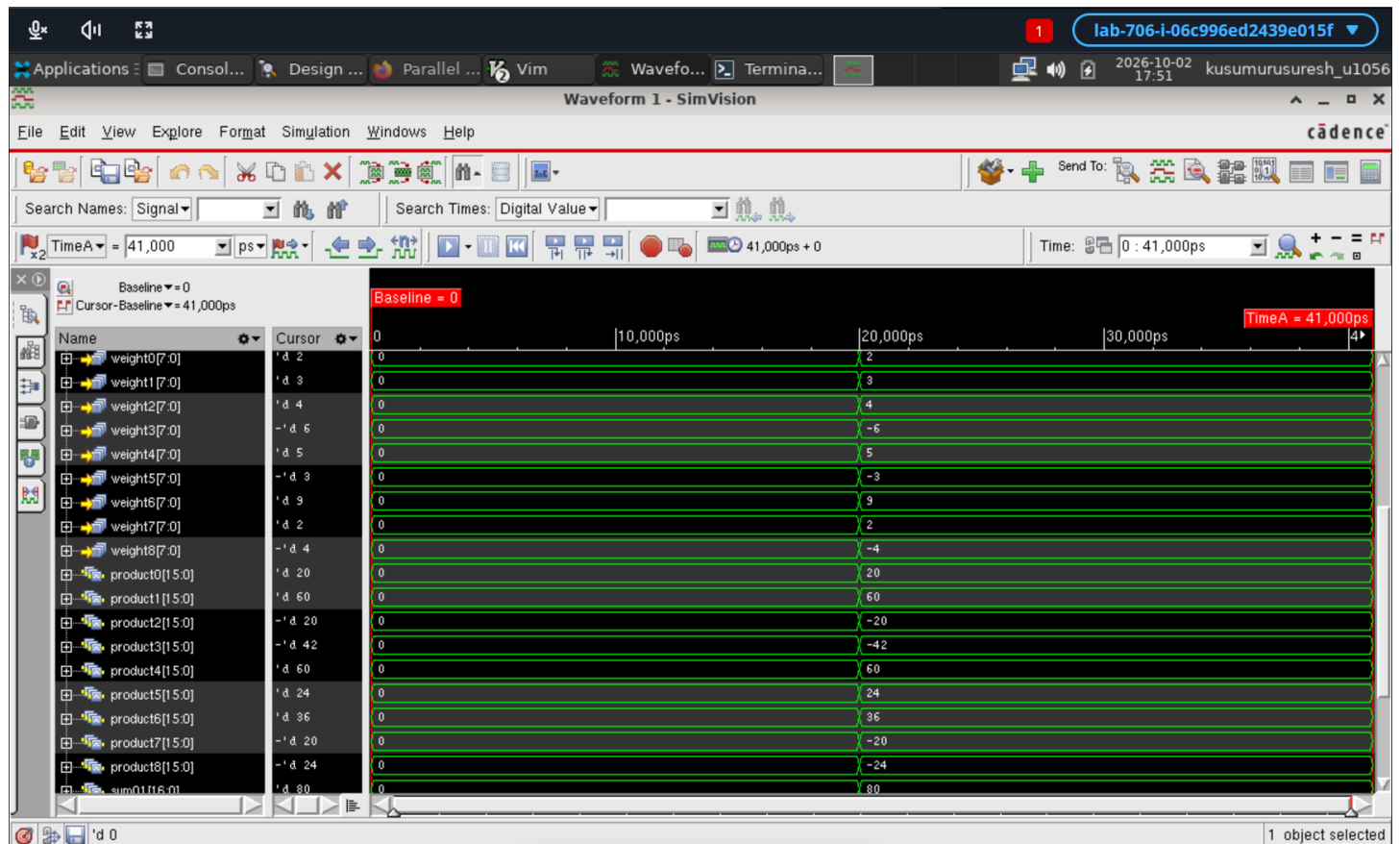
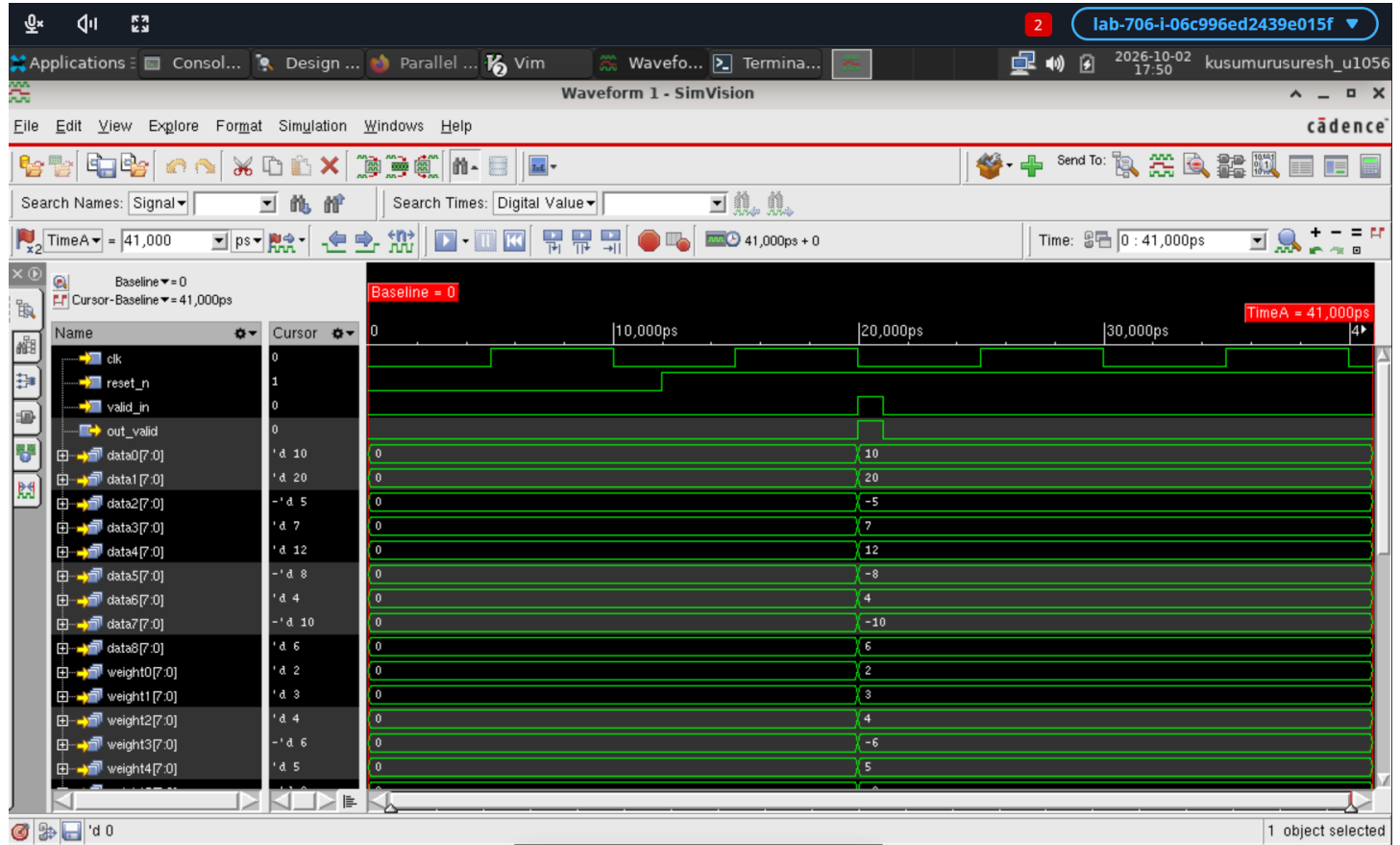
xmsim: *W,DLCPTH (/data/tools/DV/cdn/XCELIUM/tools/inca/files/cdsvhdl.lib,12): cds.lib Invalid path '/data/tools/
DV/cdn/XCELIUM/tools/inca/files/CDS_ASSERTIONS' (cds.lib command ignored).
WARNING: Can't use the home directory defined in the SIMVISION_HOME
environment variable (/data/tools/DV/cdn/XCELIUM/tools.lnx86)
because the directory is either not writeable or readable
Instead the normal home directory (/home/kusumurusuresh_u1056)
will be used.
Loading snapshot worklib.tb_mac_parallel:sv ..... Done
xcelium> source /data/tools/DV/cdn/XCELIUM/tools/xcelium/files/xmsimrc
xcelium> run
xmsim: *W,DVEXACC: some objects excluded from $dumpvars due to access restrictions, use +access+r on command line
for access to all objects.
File: ./tb_mac_parallel.sv, line = 83, pos = 16
Scope: tb_mac_parallel
Time: 0 FS + 0

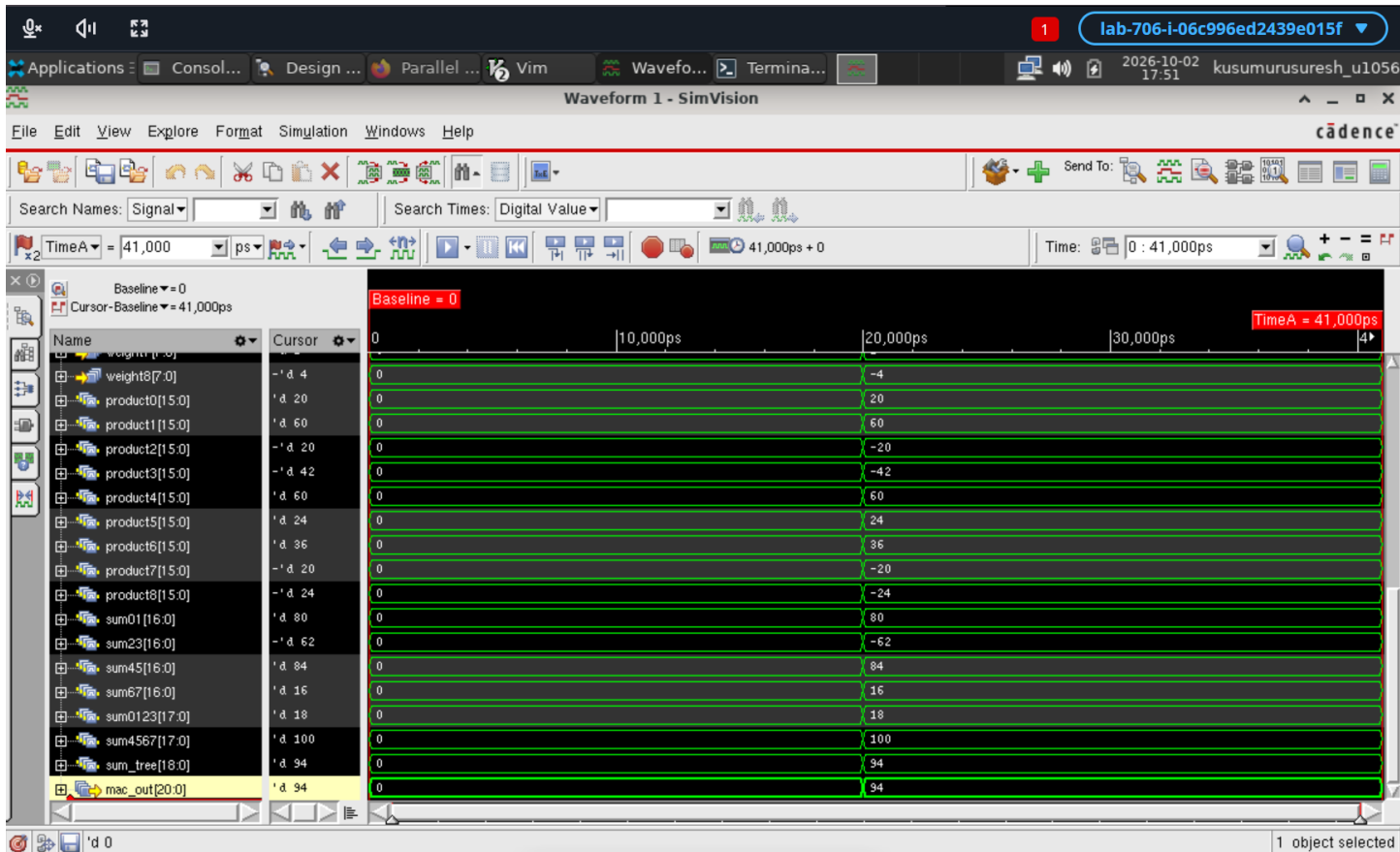
=====
1(c)(i) NON-PIPELINED TEST PASSED
=====
Expected MAC = 94
Actual MAC = 94
out_valid = 1
=====

Simulation complete via $finish(1) at time 41 NS + 0
./tb_mac_parallel.sv:181 $finish;
xcelium> exit
TOOL: xrun(64) 25.09-s009: Exiting on Oct 02, 2026 at 17:38:42 UTC (total: 00:00:05)
kusumurusuresh_u1056@lab-706-i-06c996ed2439e015f non_pipelined >

```

6. Cadence SimVision Waveform Viewer:





Pipelined_3stage:

1. mac_parallel_pipe3.sv:

```
`timescale 1ns/1ps

module mac_parallel_pipe3 #(
    parameter W = 8,
    parameter N = 9
) (
    input  logic clk,
    input  logic reset_n,
    input  logic valid_in,

    input  logic signed [W-1:0] data0,
    input  logic signed [W-1:0] data1,
    input  logic signed [W-1:0] data2,
    input  logic signed [W-1:0] data3,
    input  logic signed [W-1:0] data4,
    input  logic signed [W-1:0] data5,
    input  logic signed [W-1:0] data6,
    input  logic signed [W-1:0] data7,
    input  logic signed [W-1:0] data8,

    input  logic signed [W-1:0] weight0,
    input  logic signed [W-1:0] weight1,
    input  logic signed [W-1:0] weight2,
    input  logic signed [W-1:0] weight3,
    input  logic signed [W-1:0] weight4,
    input  logic signed [W-1:0] weight5,
    input  logic signed [W-1:0] weight6,
    input  logic signed [W-1:0] weight7,
    input  logic signed [W-1:0] weight8,

    output logic signed [2*W+4:0] mac_out,
    output logic out_valid
);

    // =====
    // STAGE 1
    // Nine parallel multipliers
    // =====

    logic signed [2*W-1:0] product0_s1;
    logic signed [2*W-1:0] product1_s1;
    logic signed [2*W-1:0] product2_s1;
    logic signed [2*W-1:0] product3_s1;
    logic signed [2*W-1:0] product4_s1;
    logic signed [2*W-1:0] product5_s1;
    logic signed [2*W-1:0] product6_s1;
    logic signed [2*W-1:0] product7_s1;
    logic signed [2*W-1:0] product8_s1;

    // =====
    // STAGE 2
    // Partial sums
```

```

// =====

logic signed [2*W:0] sum01_s2;
logic signed [2*W:0] sum23_s2;
logic signed [2*W:0] sum45_s2;
logic signed [2*W:0] sum67_s2;

logic signed [2*W+1:0] sum0123_s2;
logic signed [2*W+1:0] sum4567_s2;

logic signed [2*W-1:0] product8_s2;

// =====
// STAGE 3
// Final result
// =====

logic signed [2*W+2:0] sum_tree_s3;

// Valid pipeline
logic valid_s1;
logic valid_s2;
logic valid_s3;

// =====
// Sequential pipeline
// =====

always_ff @(posedge clk) begin

    if (!reset_n) begin

        // Stage 1
        product0_s1 <= '0;
        product1_s1 <= '0;
        product2_s1 <= '0;
        product3_s1 <= '0;
        product4_s1 <= '0;
        product5_s1 <= '0;
        product6_s1 <= '0;
        product7_s1 <= '0;
        product8_s1 <= '0;

        // Stage 2
        sum01_s2 <= '0;
        sum23_s2 <= '0;
        sum45_s2 <= '0;
        sum67_s2 <= '0;
        sum0123_s2 <= '0;
        sum4567_s2 <= '0;
        product8_s2 <= '0;

        // Stage 3
        sum_tree_s3 <= '0;

```

```

// Valid pipeline
valid_s1 <= 1'b0;
valid_s2 <= 1'b0;
valid_s3 <= 1'b0;

mac_out    <= '0;
out_valid <= 1'b0;

end

else begin

    // =====
    // STAGE 1
    // =====

    valid_s1 <= valid_in;

    if (valid_in) begin

        product0_s1 <= data0 * weight0;
        product1_s1 <= data1 * weight1;
        product2_s1 <= data2 * weight2;
        product3_s1 <= data3 * weight3;
        product4_s1 <= data4 * weight4;
        product5_s1 <= data5 * weight5;
        product6_s1 <= data6 * weight6;
        product7_s1 <= data7 * weight7;
        product8_s1 <= data8 * weight8;

    end

    // =====
    // STAGE 2
    // =====

    valid_s2 <= valid_s1;

    if (valid_s1) begin

        sum01_s2 <= product0_s1 + product1_s1;
        sum23_s2 <= product2_s1 + product3_s1;
        sum45_s2 <= product4_s1 + product5_s1;
        sum67_s2 <= product6_s1 + product7_s1;

        sum0123_s2 <=
            (product0_s1 + product1_s1) +
            (product2_s1 + product3_s1);

        sum4567_s2 <=
            (product4_s1 + product5_s1) +
            (product6_s1 + product7_s1);

        product8_s2 <= product8_s1;
    end
end

```

```

        end

        // =====
        // STAGE 3
        // =====

        valid_s3 <= valid_s2;

        if (valid_s2) begin

            sum_tree_s3 <=
                sum0123_s2 +
                sum4567_s2 +
                product8_s2;

            mac_out <=
                sum0123_s2 +
                sum4567_s2 +
                product8_s2;

        end

        out_valid <= valid_s2;

    end

end

endmodule

```

2.tb_mac_parallel_pipe3.sv

```

`timescale 1ns/1ps

module tb_mac_parallel_pipe3;

    logic clk;
    logic reset_n;
    logic valid_in;

    logic signed [7:0] data0;
    logic signed [7:0] data1;
    logic signed [7:0] data2;
    logic signed [7:0] data3;
    logic signed [7:0] data4;
    logic signed [7:0] data5;
    logic signed [7:0] data6;
    logic signed [7:0] data7;
    logic signed [7:0] data8;

    logic signed [7:0] weight0;
    logic signed [7:0] weight1;
    logic signed [7:0] weight2;
    logic signed [7:0] weight3;
    logic signed [7:0] weight4;
    logic signed [7:0] weight5;

```

```

logic signed [7:0] weight6;
logic signed [7:0] weight7;
logic signed [7:0] weight8;

logic signed [20:0] mac_out;
logic out_valid;

integer expected;

// =====
// DUT
// =====

mac_parallel_pipe3 dut (
    .clk(clk),
    .reset_n(reset_n),
    .valid_in(valid_in),

    .data0(data0),
    .data1(data1),
    .data2(data2),
    .data3(data3),
    .data4(data4),
    .data5(data5),
    .data6(data6),
    .data7(data7),
    .data8(data8),

    .weight0(weight0),
    .weight1(weight1),
    .weight2(weight2),
    .weight3(weight3),
    .weight4(weight4),
    .weight5(weight5),
    .weight6(weight6),
    .weight7(weight7),
    .weight8(weight8),

    .mac_out(mac_out),
    .out_valid(out_valid)
);

// =====
// CLOCK
// =====

initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

// =====
// TEST
// =====

```

```
initial begin
```

```
    $dumpfile("mac_parallel_pipe3.vcd");  
    $dumpvars(0, tb_mac_parallel_pipe3);
```

```
    reset_n = 1'b0;  
    valid_in = 1'b0;
```

```
    data0 = 0;  
    data1 = 0;  
    data2 = 0;  
    data3 = 0;  
    data4 = 0;  
    data5 = 0;  
    data6 = 0;  
    data7 = 0;  
    data8 = 0;
```

```
    weight0 = 0;  
    weight1 = 0;  
    weight2 = 0;  
    weight3 = 0;  
    weight4 = 0;  
    weight5 = 0;  
    weight6 = 0;  
    weight7 = 0;  
    weight8 = 0;
```

```
    expected = 94;
```

```
    // =====  
    // RESET  
    // =====
```

```
    repeat (2) @(posedge clk);
```

```
    reset_n = 1'b1;
```

```
    // =====  
    // INPUT VECTOR  
    // =====
```

```
    @(negedge clk);
```

```
    data0 = 8'sd10;  
    data1 = 8'sd20;  
    data2 = -8'sd5;  
    data3 = 8'sd7;  
    data4 = 8'sd12;  
    data5 = -8'sd8;  
    data6 = 8'sd4;  
    data7 = -8'sd10;  
    data8 = 8'sd6;
```

```
    weight0 = 8'sd2;
```

```

weight1 = 8'sd3;
weight2 = 8'sd4;
weight3 = -8'sd6;
weight4 = 8'sd5;
weight5 = -8'sd3;
weight6 = 8'sd9;
weight7 = 8'sd2;
weight8 = -8'sd4;

valid_in = 1'b1;

// Stage 1
@(posedge clk);

// Remove valid after one input vector
@(negedge clk);
valid_in = 1'b0;

// =====
// Wait for 3-stage pipeline
// =====

repeat (2) @(posedge clk);

#1;

// =====
// CHECK
// =====

if (out_valid && (mac_out == expected)) begin

    $display("");
    $display("=====");
    $display("          1(c)(ii) 3-STAGE PIPELINE PASSED");
    $display("=====");
    $display("Expected MAC = %0d", expected);
    $display("Actual MAC   = %0d", mac_out);
    $display("out_valid    = %b", out_valid);
    $display("=====");
    $display("");

end
else begin

    $display("");
    $display("=====");
    $display("          1(c)(ii) 3-STAGE PIPELINE FAILED");
    $display("=====");
    $display("Expected MAC = %0d", expected);
    $display("Actual MAC   = %0d", mac_out);
    $display("out_valid    = %b", out_valid);
    $display("=====");
    $display("");

end

```



```

end

#20;
$finish;

end

endmodule

```

3. terminal:

The screenshot shows a VS Code terminal window with the following content:

```

sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1c/pipelined_3stage$ iverilog -g2012 -o mac_parallel_pipe3_sim \
mac_parallel_pipe3.v sv tb_mac_parallel_pipe3.v sv
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1c/pipelined_3stage$ vvp mac_parallel_pipe3_sim
VCD info: dumpfile mac_parallel_pipe3.vcd opened for output.

=====
1(c)(ii) 3-STAGE PIPELINE PASSED
=====
Expected MAC = 94
Actual MAC   = 94
out_valid    = 1
=====

tb_mac_parallel_pipe3.v:193: $finish called at 66000 (1ps)
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1c/pipelined_3stage$ gtkwave mac_parallel_pipe3.vcd &
[1] 33203
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1c/pipelined_3stage$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
sureshvlsi@DESKTOP-CB90UTN:~/IITKGP_assignment1/1c/pipelined_3stage$

```

The terminal window also shows the Explorer sidebar with the following files:

- Open Editors
 - Welcome
 - mac_parallel_pip...
 - tb_mac_parallel...
- pipelined_3stage [WSL...]
 - mac_parallel_pipe3.s...
 - mac_parallel_pipe3.v sv
 - mac_parallel_pipe3.v...
 - tb_mac_parallel_pipe...

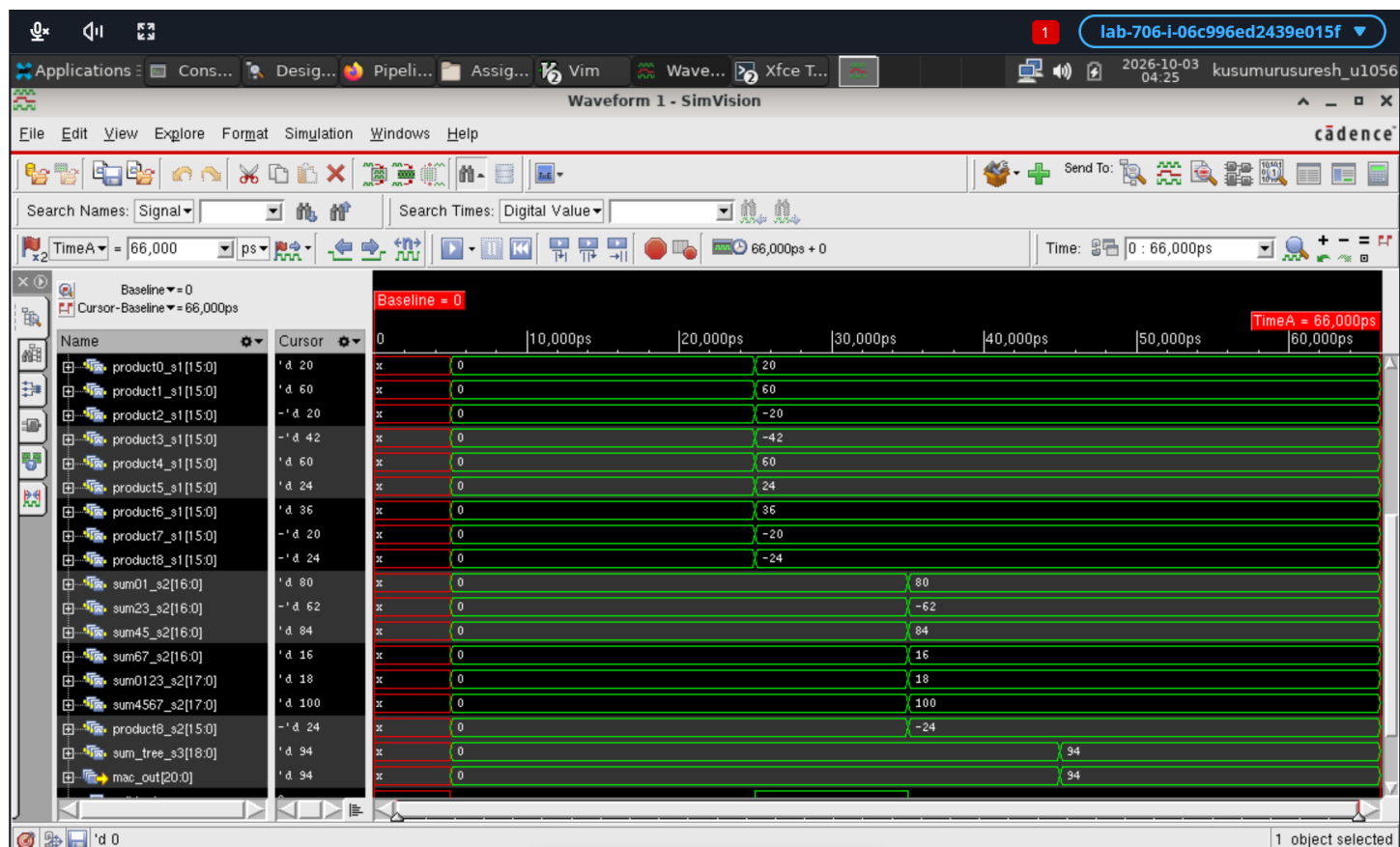
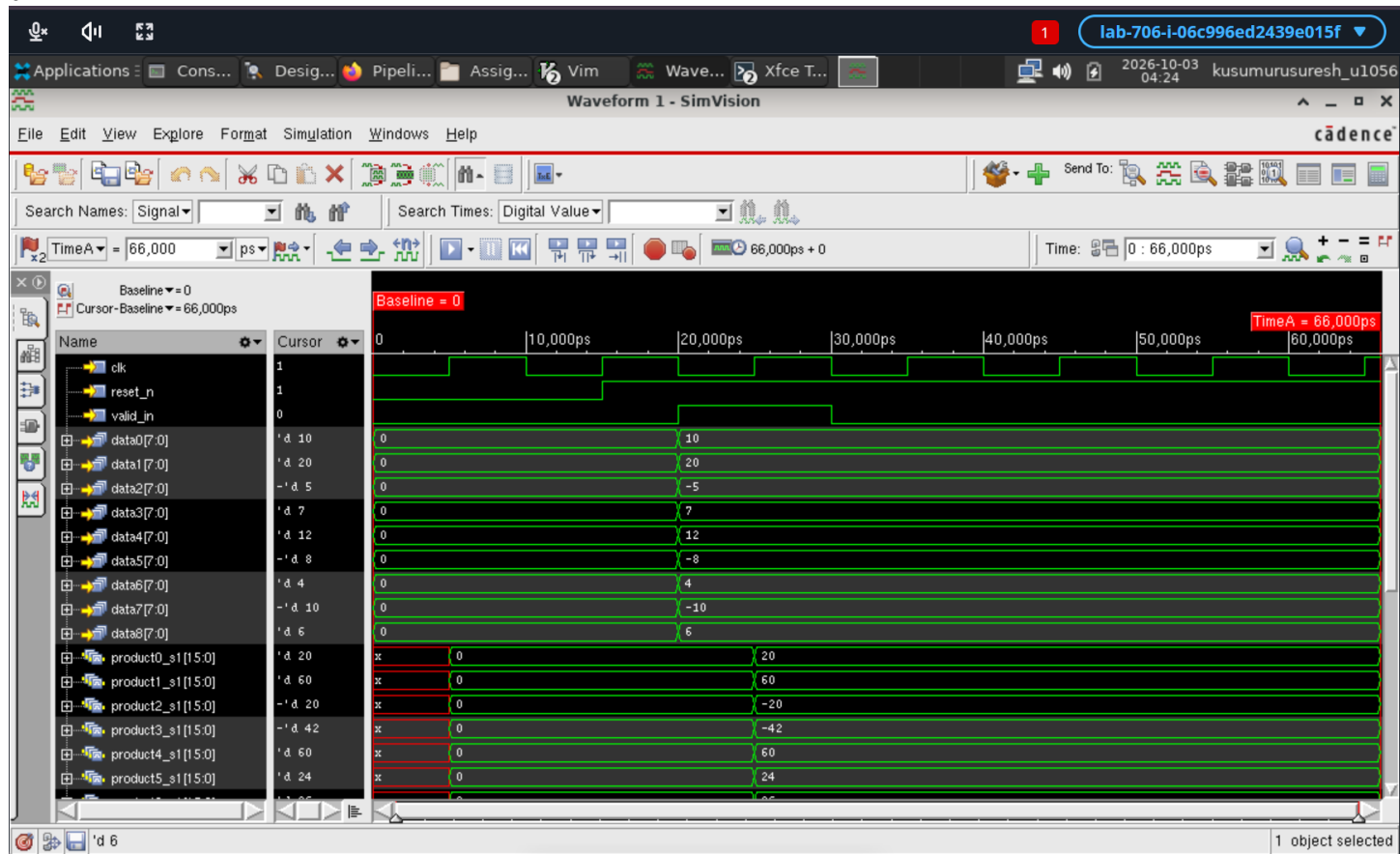
The status bar at the bottom indicates the current file is "Ln 197, Col 10" in "UTF-8" encoding, "Plain Text" format, and the system clock shows "11:01 AM 9/26/2026".

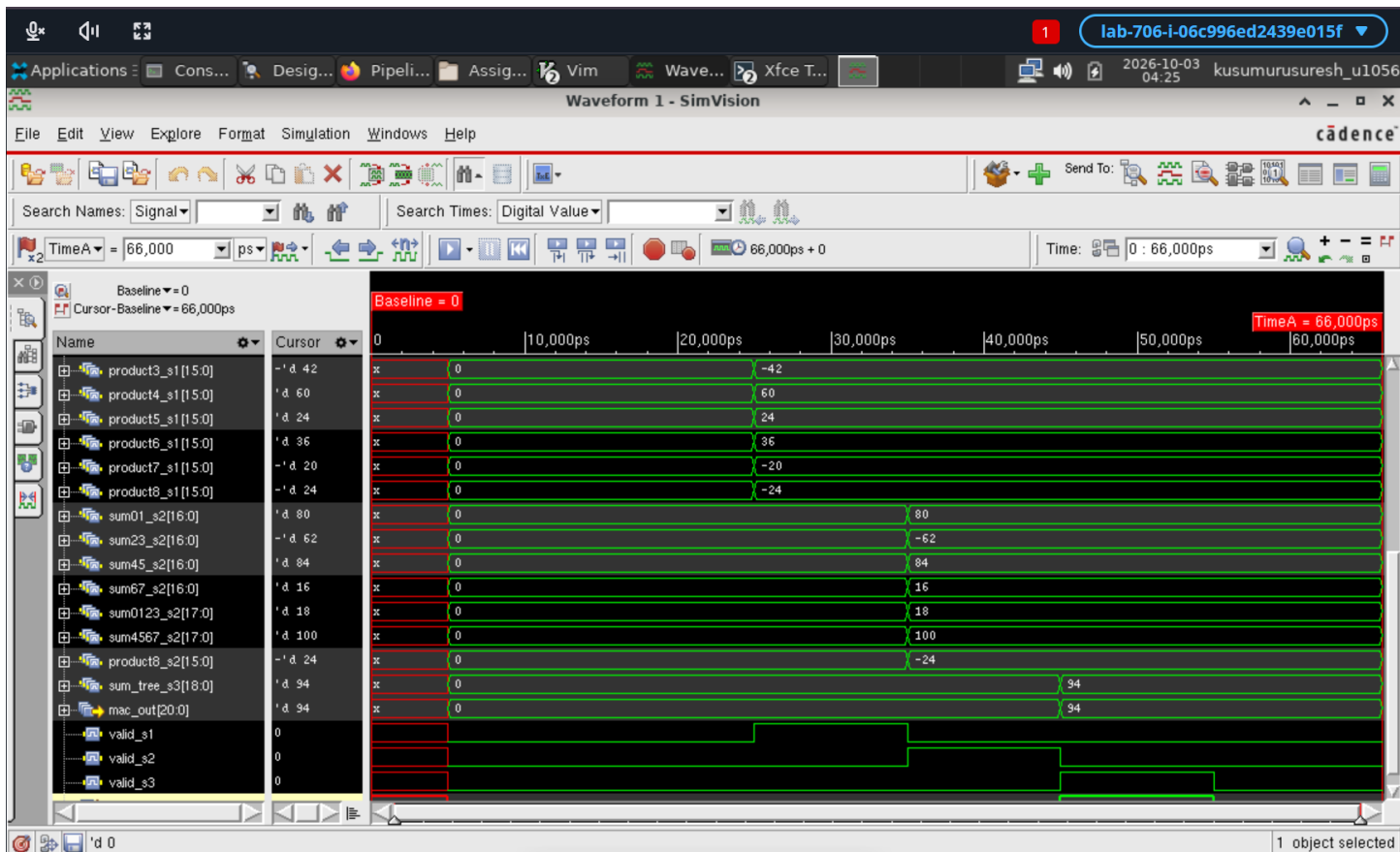
```
Applications Pipelined MAC ... Assignment_1... Vim Xfce Terminal
Terminal -
File Edit View Terminal Tabs Help
xmsim: *W,DLCPTH (/data/tools/DV/cdn/XCELIUM/tools/inca/files/cdsvhdl.lib,12): cds.lib Invalid path '/data/tools/
DV/cdn/XCELIUM/tools/inca/files/CDS_ASSERTIONS' (cds.lib command ignored).
WARNING: Can't use the home directory defined in the SIMVISION_HOME
environment variable (/data/tools/DV/cdn/XCELIUM/tools.lnx86)
because the directory is either not writeable or readable
Instead the normal home directory (/home/kusumurusuresh_u1056)
will be used.
Loading snapshot worklib.tb_mac_parallel_pipe3:sv ..... Done
xcelium> source /data/tools/DV/cdn/XCELIUM/tools/xcelium/files/xmsimrc
xcelium> run
xmsim: *W,DVEXACC: some objects excluded from $dumpvars due to access restrictions, use +access+r on command line
for access to all objects.
File: ./tb_mac_parallel_pipe3.sv, line = 83, pos = 16
Scope: tb_mac_parallel_pipe3
Time: 0 FS + 0

=====
1(c)(ii) 3-STAGE PIPELINE PASSED
=====
Expected MAC = 94
Actual MAC = 94
out_valid = 1
=====

Simulation complete via $finish(1) at time 66 NS + 0
./tb_mac_parallel_pipe3.sv:193 $finish;
xcelium> exit
TOOL: xrun(64) 25.09-s009: Exiting on Oct 03, 2026 at 04:16:46 UTC (total: 00:00:05)
kusumurusuresh_u1056@lab-706-i-06c996ed2439e015f pipelined_3stage >
```

6. Cadence SimVision Waveform Viewer:





Assignment 2(i) — Consolidated 64 KB SRAM:

1. sram_64kb.sv:

```
`timescale 1ns/1ps

module sram_64kb (
    input logic      clk,
    input logic      we,
    input logic      re,
    input logic [13:0] addr,
    input logic [31:0] wdata,
    output logic [31:0] rdata
);

    // =====
    // 64 KB SRAM
    //
    // Physical array:
    //     1024 rows x 512 bits
    //
    // 512 bits / 32 bits = 16 words per row
    //
    // Total:
    //     1024 x 16 = 16384 words
    //     16384 x 4 bytes = 65536 bytes = 64 KB
    // =====

    logic [511:0] mem [0:1023];

    logic [9:0] row_addr;
    logic [3:0] word_sel;

    assign row_addr = addr[13:4];
    assign word_sel = addr[3:0];

    // =====
    // WRITE
    // =====

    always_ff @(posedge clk) begin

        if (we) begin

            mem[row_addr][word_sel*32 +: 32] <= wdata;

        end

    end

    // =====
    // READ
    // =====

    always_comb begin
```

```

        if (re) begin
            rdata = mem[row_addr][word_sel*32 +: 32];

        end
        else begin

            rdata = 32'b0;

        end

    end

end
endmodule

```

2. tb_sram_64kb.sv:

```

`timescale 1ns/1ps

module tb_sram_64kb;

    logic        clk;
    logic        we;
    logic        re;
    logic [13:0] addr;
    logic [31:0] wdata;
    logic [31:0] rdata;

    integer i;
    integer errors;

    // -----
    // Test addresses
    // -----

    logic [13:0] test_addr [0:7];
    logic [31:0] test_data [0:7];

    // -----
    // DUT
    // -----

    sram_64kb dut (
        .clk    (clk),
        .we     (we),
        .re     (re),
        .addr   (addr),
        .wdata  (wdata),
        .rdata  (rdata)
    );

    // -----
    // Clock
    // -----

    initial begin

```

```

    clk = 1'b0;

    forever #5 clk = ~clk;
end

// -----
// Test
// -----

initial begin

    $dumpfile("sram_64kb.vcd");
    $dumpvars(0, tb_sram_64kb);

    we = 1'b0;
    re = 1'b0;
    addr = 14'd0;
    wdata = 32'd0;

    errors = 0;

    // -----
    // Eight test locations
    // -----

    test_addr[0] = 14'd0;
    test_addr[1] = 14'd1;
    test_addr[2] = 14'd15;
    test_addr[3] = 14'd16;
    test_addr[4] = 14'd100;
    test_addr[5] = 14'd1023;
    test_addr[6] = 14'd4095;
    test_addr[7] = 14'd16383;

    test_data[0] = 32'hA5A50001;
    test_data[1] = 32'hA5A50002;
    test_data[2] = 32'hA5A50003;
    test_data[3] = 32'hA5A50004;
    test_data[4] = 32'hA5A50005;
    test_data[5] = 32'hA5A50006;
    test_data[6] = 32'hA5A50007;
    test_data[7] = 32'hA5A50008;

    // -----
    // RESET / INITIALIZATION
    // -----

    #10;

    // =====
    // WRITE 8 LOCATIONS
    // =====

    $display("");
    $display("=====");

```

```

$display("          Q2(i) SRAM WRITE OPERATION");
$display("=====");

we = 1'b1;
re = 1'b0;

for (i = 0; i < 8; i = i + 1) begin

    @(negedge clk);

    addr = test_addr[i];
    wdata = test_data[i];

    @(posedge clk);

    #1;

    $display(
        "WRITE | addr=%0d | row=%0d | word=%0d | data=0x%08h",
        addr,
        addr >> 4,
        addr & 14'h000F,
        wdata
    );

end

// Stop writing
@(negedge clk);

we = 1'b0;

// =====
// READ AND VERIFY 8 LOCATIONS
// =====

$display("");
$display("=====");
$display("          Q2(i) SRAM READ / VERIFY");
$display("=====");

re = 1'b1;

for (i = 0; i < 8; i = i + 1) begin

    @(negedge clk);

    addr = test_addr[i];

    #1;

    if (rdata === test_data[i]) begin

        $display(
            "READ | addr=%0d | expected=0x%08h | actual=0x%08h | PASS",

```



```

        addr,
        test_data[i],
        rdata
    );

end
else begin

    $display(
        "READ | addr=%0d | expected=0x%08h | actual=0x%08h | FAIL",
        addr,
        test_data[i],
        rdata
    );

    errors = errors + 1;

end

end

// =====
// FINAL RESULT
// =====

$display("");
$display("=====");

if (errors == 0) begin

    $display("    ASSIGNMENT 2(i) TEST PASSED");
    $display("    64 KB CONSOLIDATED SRAM");
    $display("    8 LOCATIONS VERIFIED");

end
else begin

    $display("    ASSIGNMENT 2(i) TEST FAILED");
    $display("    ERRORS = %0d", errors);

end

$display("=====");
$display("");

re = 1'b0;

#20;

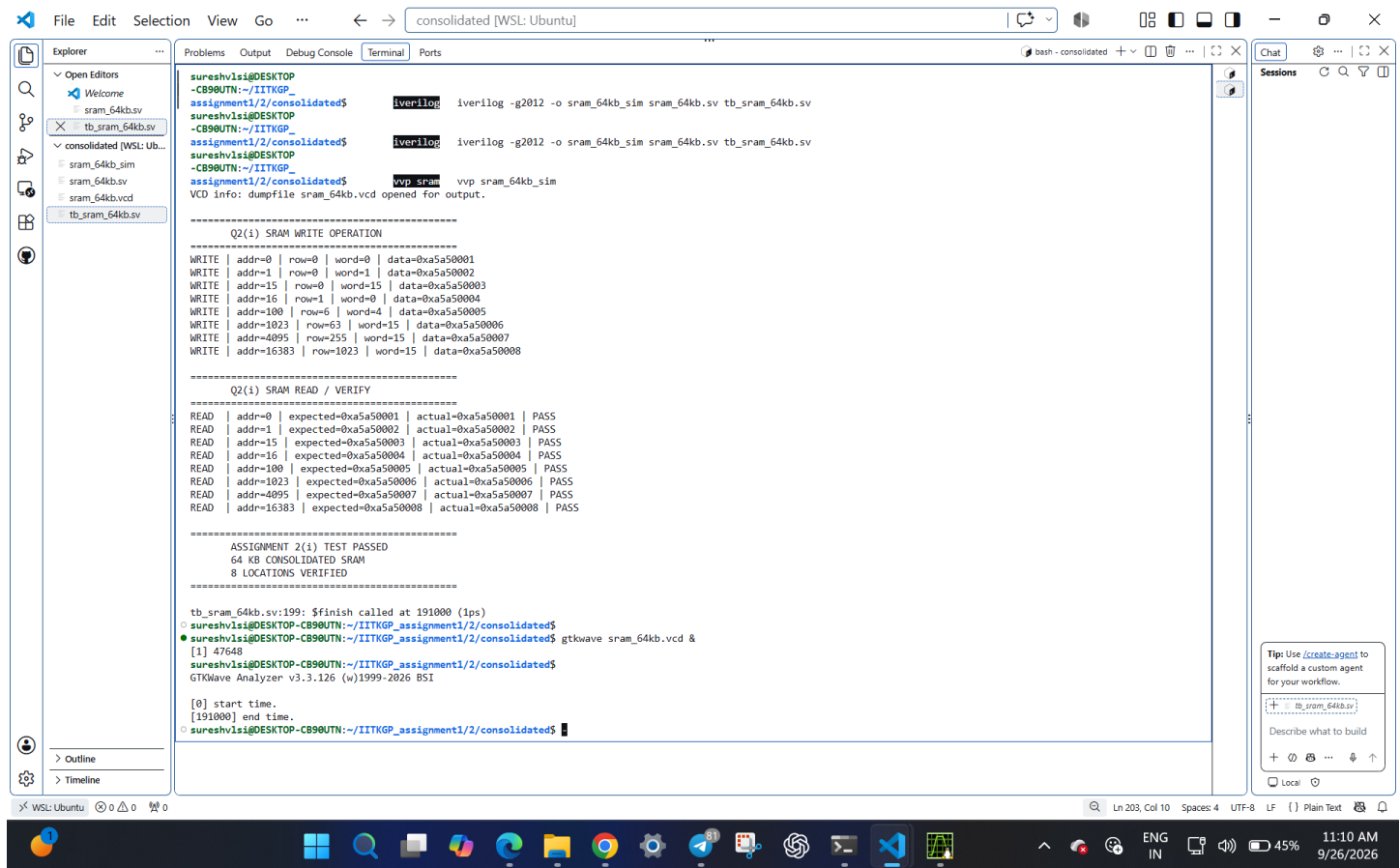
$finish;

end

endmodule

```

3. terminal:



```
File Edit Selection View Go ... consolidated [WSL: Ubuntu]
Explorer
  Open Editors
    Welcome
    sram_64kb.sv
    tb_sram_64kb.sv
  consolidated [WSL: Ubuntu]
    sram_64kb_sim
    sram_64kb.sv
    sram_64kb.vcd
    tb_sram_64kb.sv
Problems Output Debug Console Terminal Ports
bash - consolidated
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$ iverilog -g2012 -o sram_64kb_sim sram_64kb.sv tb_sram_64kb.sv
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$ iverilog -g2012 -o sram_64kb_sim sram_64kb.sv tb_sram_64kb.sv
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$ vvp sram_64kb_sim
VCD info: dumpfile sram_64kb.vcd opened for output.

-----
Q2(i) SRAM WRITE OPERATION
-----
WRITE | addr=0 | row=0 | word=0 | data=0xa5a50001
WRITE | addr=1 | row=0 | word=1 | data=0xa5a50002
WRITE | addr=15 | row=0 | word=15 | data=0xa5a50003
WRITE | addr=16 | row=1 | word=0 | data=0xa5a50004
WRITE | addr=100 | row=6 | word=4 | data=0xa5a50005
WRITE | addr=1023 | row=63 | word=15 | data=0xa5a50006
WRITE | addr=4095 | row=255 | word=15 | data=0xa5a50007
WRITE | addr=16383 | row=1023 | word=15 | data=0xa5a50008

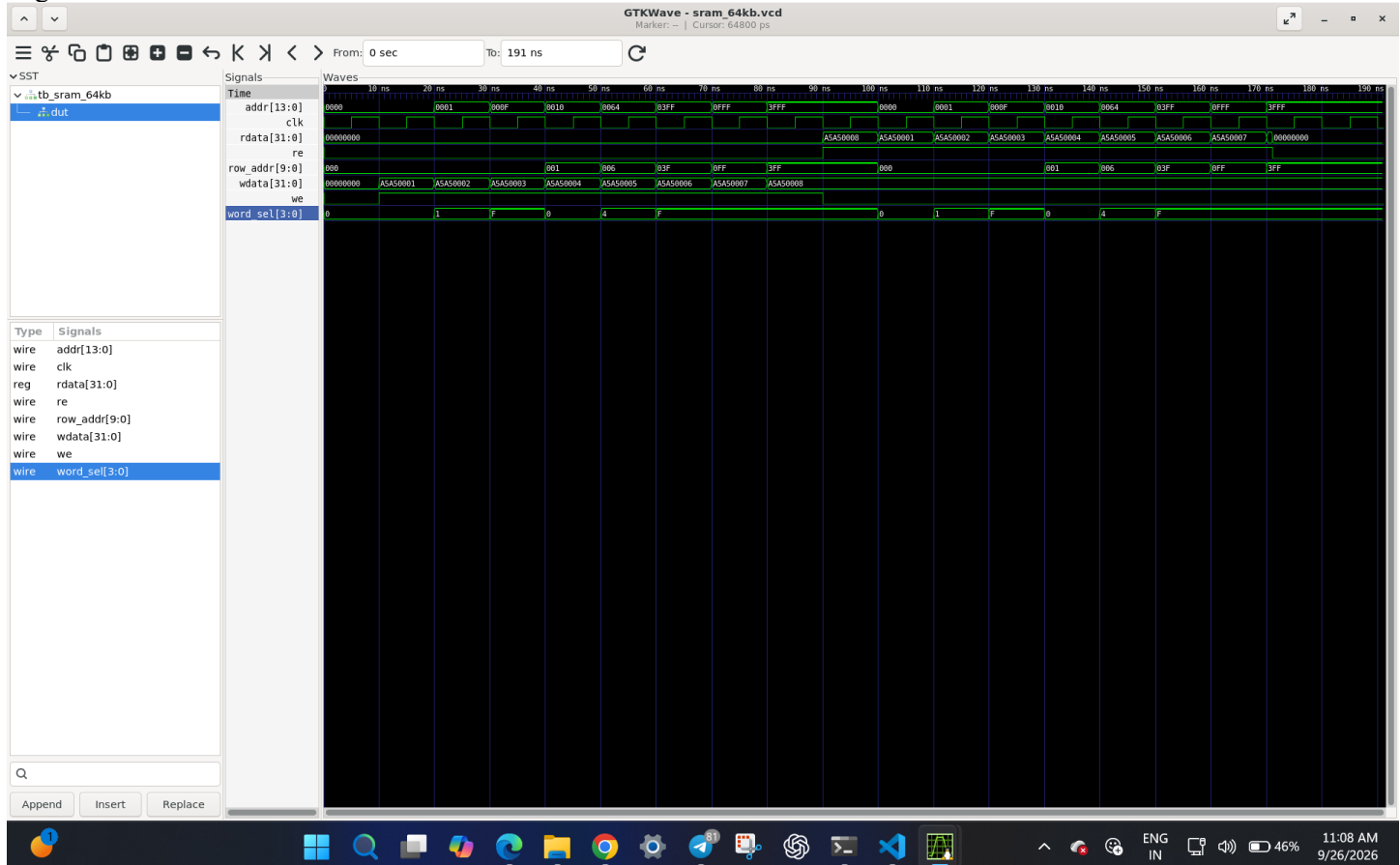
-----
Q2(i) SRAM READ / VERIFY
-----
READ | addr=0 | expected=0xa5a50001 | actual=0xa5a50001 | PASS
READ | addr=1 | expected=0xa5a50002 | actual=0xa5a50002 | PASS
READ | addr=15 | expected=0xa5a50003 | actual=0xa5a50003 | PASS
READ | addr=16 | expected=0xa5a50004 | actual=0xa5a50004 | PASS
READ | addr=100 | expected=0xa5a50005 | actual=0xa5a50005 | PASS
READ | addr=1023 | expected=0xa5a50006 | actual=0xa5a50006 | PASS
READ | addr=4095 | expected=0xa5a50007 | actual=0xa5a50007 | PASS
READ | addr=16383 | expected=0xa5a50008 | actual=0xa5a50008 | PASS

-----
ASSIGNMENT 2(i) TEST PASSED
64 KB CONSOLIDATED SRAM
8 LOCATIONS VERIFIED
-----

tb_sram_64kb.sv:199: $finish called at 191000 (1ps)
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$ gtwave sram_64kb.vcd &
[1] 47648
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[191000] end time.
suresh@lsid@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/consolidated$
```

4. gtwave:



5. Cadence Xcelium simulation environment terminal:

```
lab-706-i-06c996ed2439e015f
Applications Explain SRAM Design ... Vim Terminal - 2026-10-03 04:48 ksumurusuresh_u1056
Terminal -
File Edit View Terminal Tabs Help

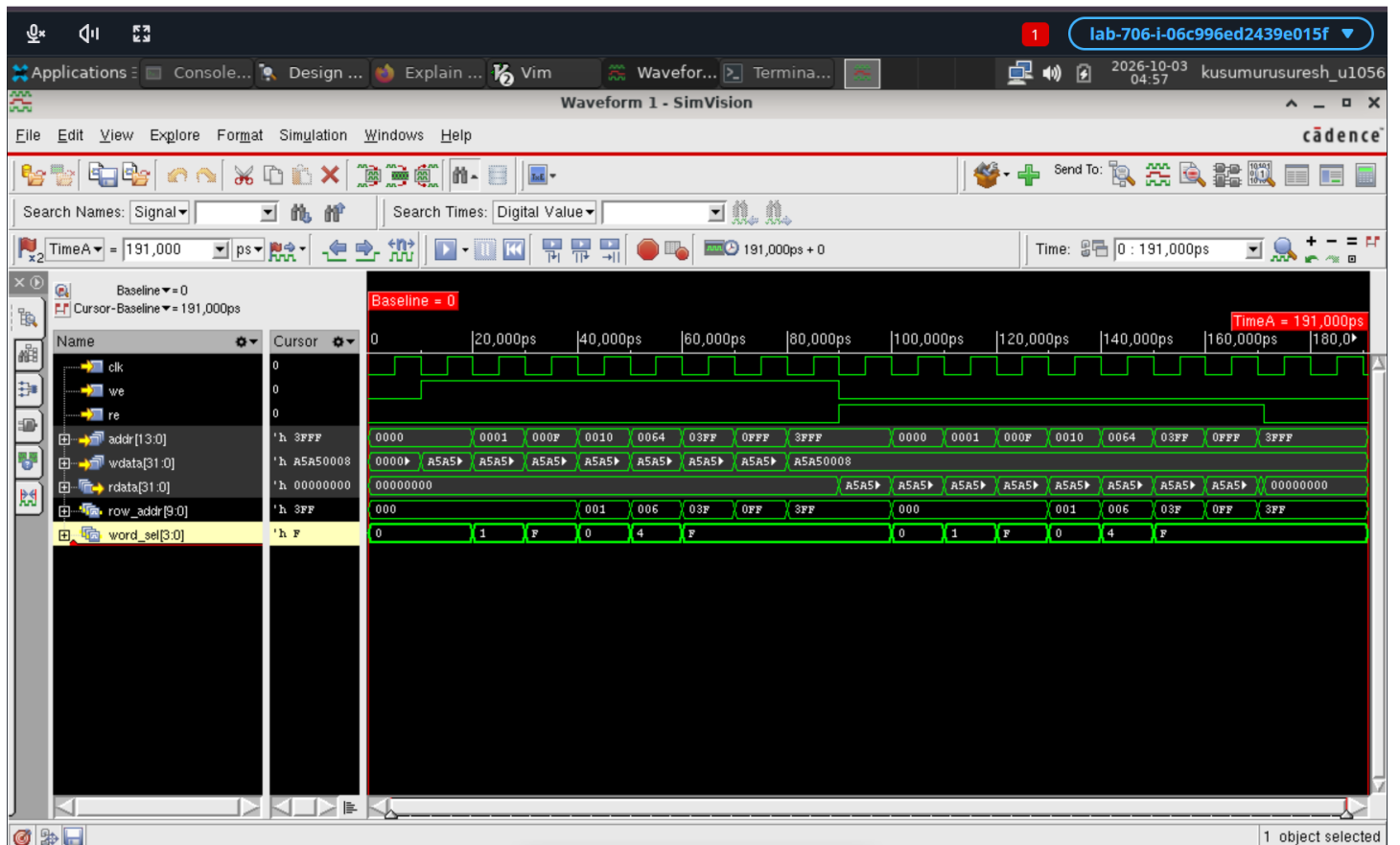
=====
Q2(i) SRAM WRITE OPERATION
=====
WRITE | addr=0 | row=0 | word=0 | data=0xa5a50001
WRITE | addr=1 | row=0 | word=1 | data=0xa5a50002
WRITE | addr=15 | row=0 | word=15 | data=0xa5a50003
WRITE | addr=16 | row=1 | word=0 | data=0xa5a50004
WRITE | addr=100 | row=6 | word=4 | data=0xa5a50005
WRITE | addr=1023 | row=63 | word=15 | data=0xa5a50006
WRITE | addr=4095 | row=255 | word=15 | data=0xa5a50007
WRITE | addr=16383 | row=1023 | word=15 | data=0xa5a50008
=====

Q2(i) SRAM READ / VERIFY
=====
READ | addr=0 | expected=0xa5a50001 | actual=0xa5a50001 | PASS
READ | addr=1 | expected=0xa5a50002 | actual=0xa5a50002 | PASS
READ | addr=15 | expected=0xa5a50003 | actual=0xa5a50003 | PASS
READ | addr=16 | expected=0xa5a50004 | actual=0xa5a50004 | PASS
READ | addr=100 | expected=0xa5a50005 | actual=0xa5a50005 | PASS
READ | addr=1023 | expected=0xa5a50006 | actual=0xa5a50006 | PASS
READ | addr=4095 | expected=0xa5a50007 | actual=0xa5a50007 | PASS
READ | addr=16383 | expected=0xa5a50008 | actual=0xa5a50008 | PASS
=====

ASSIGNMENT 2(i) TEST PASSED
64 KB CONSOLIDATED SRAM
8 LOCATIONS VERIFIED
=====

Simulation complete via $finish(1) at time 191 NS + 0
./tb_sram_64kb.sv:199 $finish;
xcelium> exit
T00L: xrun(64) 25.09-s009; Exiting on Oct 03, 2026 at 04:46:45 UTC (total: 00:00:04)
ksumurusuresh_u1056@lab-706-i-06c996ed2439e015f consolidated >
```

6. Cadence SimVision Waveform Viewer:





Assignment 2(ii) — 4-Bank SRAM

1. sram_4banks.sv:

```
`timescale 1ns/1ps

module sram_4banks (
    input logic      clk,
    input logic      we,
    input logic      re,

    // 14-bit 32-bit-word address
    //
    // [13:12] = bank select
    // [11:4]  = row address
    // [3:0]   = word select within 512-bit row
    input logic [13:0] addr,

    input logic [31:0] wdata,
    output logic [31:0] rdata,

    output logic [1:0] bank_sel
);

    // =====
    // FOUR-BANK SRAM
    //
    // Each bank:
    //     256 rows x 512 bits
    //
    // Each 512-bit row contains:
    //     512 / 32 = 16 words
    //
    // Therefore:
    //     256 x 16 = 4096 words
    //
    // Each bank:
    //     4096 x 4 bytes = 16384 bytes = 16 KB
    //
    // Four banks:
    //     4 x 16 KB = 64 KB
    // =====

    logic [511:0] bank0 [0:255];
    logic [511:0] bank1 [0:255];
    logic [511:0] bank2 [0:255];
    logic [511:0] bank3 [0:255];

    // =====
    // ADDRESS DECODING
    // =====

    logic [7:0] row_addr;
    logic [3:0] word_sel;

    assign bank_sel = addr[13:12];
```

```

assign row_addr = addr[11:4];
assign word_sel = addr[3:0];

// =====
// WRITE OPERATION
// =====

always_ff @(posedge clk) begin

    if (we) begin

        case (bank_sel)

            2'd0: begin
                bank0[row_addr][word_sel*32 +: 32] <= wdata;
            end

            2'd1: begin
                bank1[row_addr][word_sel*32 +: 32] <= wdata;
            end

            2'd2: begin
                bank2[row_addr][word_sel*32 +: 32] <= wdata;
            end

            2'd3: begin
                bank3[row_addr][word_sel*32 +: 32] <= wdata;
            end

        endcase

    end

end

// =====
// READ OPERATION
// =====

always_comb begin

    rdata = 32'b0;

    if (re) begin

        case (bank_sel)

            2'd0: begin
                rdata = bank0[row_addr][word_sel*32 +: 32];
            end

            2'd1: begin
                rdata = bank1[row_addr][word_sel*32 +: 32];
            end

        end
    end
end

```

```

                2'd2: begin
                    rdata = bank2[row_addr][word_sel*32 +: 32];
                end

                2'd3: begin
                    rdata = bank3[row_addr][word_sel*32 +: 32];
                end

            endcase

        end

    end

endmodule

```

2.tb_sram_4banks.sv:

```

`timescale 1ns/1ps

module tb_sram_4banks;

    logic        clk;
    logic        we;
    logic        re;
    logic [13:0] addr;
    logic [31:0] wdata;
    logic [31:0] rdata;
    logic [1:0]  bank_sel;

    integer i;
    integer errors;

    logic [13:0] test_addr [0:3];
    logic [31:0] test_data [0:3];

    // =====
    // DUT
    // =====

    sram_4banks dut (
        .clk      (clk),
        .we       (we),
        .re       (re),
        .addr     (addr),
        .wdata     (wdata),
        .rdata     (rdata),
        .bank_sel (bank_sel)
    );

    // =====
    // CLOCK
    // =====

    initial begin
        clk = 1'b0;
    end

```

```

    forever #5 clk = ~clk;
end

// =====
// TEST
// =====

initial begin

    $dumpfile("sram_4banks.vcd");
    $dumpvars(0, tb_sram_4banks);

    we    = 1'b0;
    re    = 1'b0;
    addr  = 14'd0;
    wdata = 32'd0;

    errors = 0;

    // =====
    // ONE LOCATION IN EACH BANK
    //
    // Same row and word position:
    //
    // Bank 0 -> 0x0005
    // Bank 1 -> 0x1005
    // Bank 2 -> 0x2005
    // Bank 3 -> 0x3005
    // =====

    test_addr[0] = 14'h0005;
    test_addr[1] = 14'h1005;
    test_addr[2] = 14'h2005;
    test_addr[3] = 14'h3005;

    test_data[0] = 32'h11111111;
    test_data[1] = 32'h22222222;
    test_data[2] = 32'h33333333;
    test_data[3] = 32'h44444444;

    #10;

    // =====
    // WRITE
    // =====

    $display("");
    $display("=====");
    $display("          Q2(ii) FOUR-BANK SRAM WRITE");
    $display("=====");

    we = 1'b1;
    re = 1'b0;

    for (i = 0; i < 4; i = i + 1) begin

```



```

        @(negedge clk);

        addr  = test_addr[i];
        wdata = test_data[i];

        @(posedge clk);

        #1;

        $display(
            "WRITE | bank=%0d | addr=0x%04h | row=%0d | word=%0d | data=0x%08h",
            bank_sel,
            addr,
            addr[11:4],
            addr[3:0],
            wdata
        );

    end

    @(negedge clk);

    we = 1'b0;

    // =====
    // READ AND VERIFY
    // =====

    $display("");
    $display("=====");
    $display("          Q2(ii) FOUR-BANK SRAM READ");
    $display("=====");

    re = 1'b1;

    for (i = 0; i < 4; i = i + 1) begin

        @(negedge clk);

        addr = test_addr[i];

        #1;

        if (rdata == test_data[i]) begin

            $display(
                "READ | bank=%0d | addr=0x%04h | expected=0x%08h | actual=0x%08h |
PASS",
                bank_sel,
                addr,
                test_data[i],
                rdata
            );
        end
    end

```

```

        end
        else begin

            $display(
                "READ  | bank=%0d | addr=0x%04h | expected=0x%08h | actual=0x%08h |
FAIL",
                bank_sel,
                addr,
                test_data[i],
                rdata
            );

            errors = errors + 1;

        end

    end

    // =====
    // FINAL RESULT
    // =====

    $display("");
    $display("=====");

    if (errors == 0) begin

        $display("      ASSIGNMENT 2(ii) TEST PASSED");
        $display("      ALL 4 BANKS VERIFIED");

    end
    else begin

        $display("      ASSIGNMENT 2(ii) TEST FAILED");
        $display("      ERRORS = %0d", errors);

    end

    $display("=====");
    $display("");

    re = 1'b0;

    #20;

    $finish;

end

endmodule

```

3. terminal:

The terminal window shows the execution of a Verilog simulation. The user runs `iverilog -g2012 -o sram_4banks_sim sram_4banks.v tb_sram_4banks.v` and `vvp sram_4banks.vvp`. The output shows the simulation results for a 4-bank SRAM, including write and read operations. The simulation passes all tests.

```
suresh@ls1@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/4banks$ iverilog -g2012 -o sram_4banks_sim sram_4banks.v tb_sram_4banks.v
suresh@ls1@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/4banks$ vvp sram_4banks.vvp
VCD info: dumpfile sram_4banks.vcd opened for output.

=====
Q2(ii) FOUR-BANK SRAM WRITE
=====
WRITE | bank=0 | addr=0x0005 | row=0 | word=5 | data=0x11111111
WRITE | bank=1 | addr=0x1005 | row=0 | word=5 | data=0x22222222
WRITE | bank=2 | addr=0x2005 | row=0 | word=5 | data=0x33333333
WRITE | bank=3 | addr=0x3005 | row=0 | word=5 | data=0x44444444
=====
Q2(ii) FOUR-BANK SRAM READ
=====
READ | bank=0 | addr=0x0005 | expected=0x11111111 | actual=0x11111111 | PASS
READ | bank=1 | addr=0x1005 | expected=0x22222222 | actual=0x22222222 | PASS
READ | bank=2 | addr=0x2005 | expected=0x33333333 | actual=0x33333333 | PASS
READ | bank=3 | addr=0x3005 | expected=0x44444444 | actual=0x44444444 | PASS
=====
ASSIGNMENT 2(ii) TEST PASSED
ALL 4 BANKS VERIFIED
=====

tb_sram_4banks.v:192: $finish called at 111000 (1ps)
suresh@ls1@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/4banks$ gtwagtkwave sram_4banks.vcd &
[1] 54350
suresh@ls1@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/4banks$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI
suresh@ls1@DESKTOP-CB90UTN:~/IITKGP_assignment1/2/4banks$
```

The output shows the simulation results for a 4-bank SRAM, including write and read operations. The simulation passes all tests.

4. gtwave:

The GTKWave waveform viewer shows the simulation results for a 4-bank SRAM. The signals are plotted against time, showing the write and read operations. The signals include `addr[13:0]`, `bank_sel[1:0]`, `clk`, `rdata[31:0]`, `re`, `row_addr[7:0]`, `wdata[31:0]`, `we`, and `word_sel[3:0]`.

The signals are plotted against time, showing the write and read operations. The signals include `addr[13:0]`, `bank_sel[1:0]`, `clk`, `rdata[31:0]`, `re`, `row_addr[7:0]`, `wdata[31:0]`, `we`, and `word_sel[3:0]`.

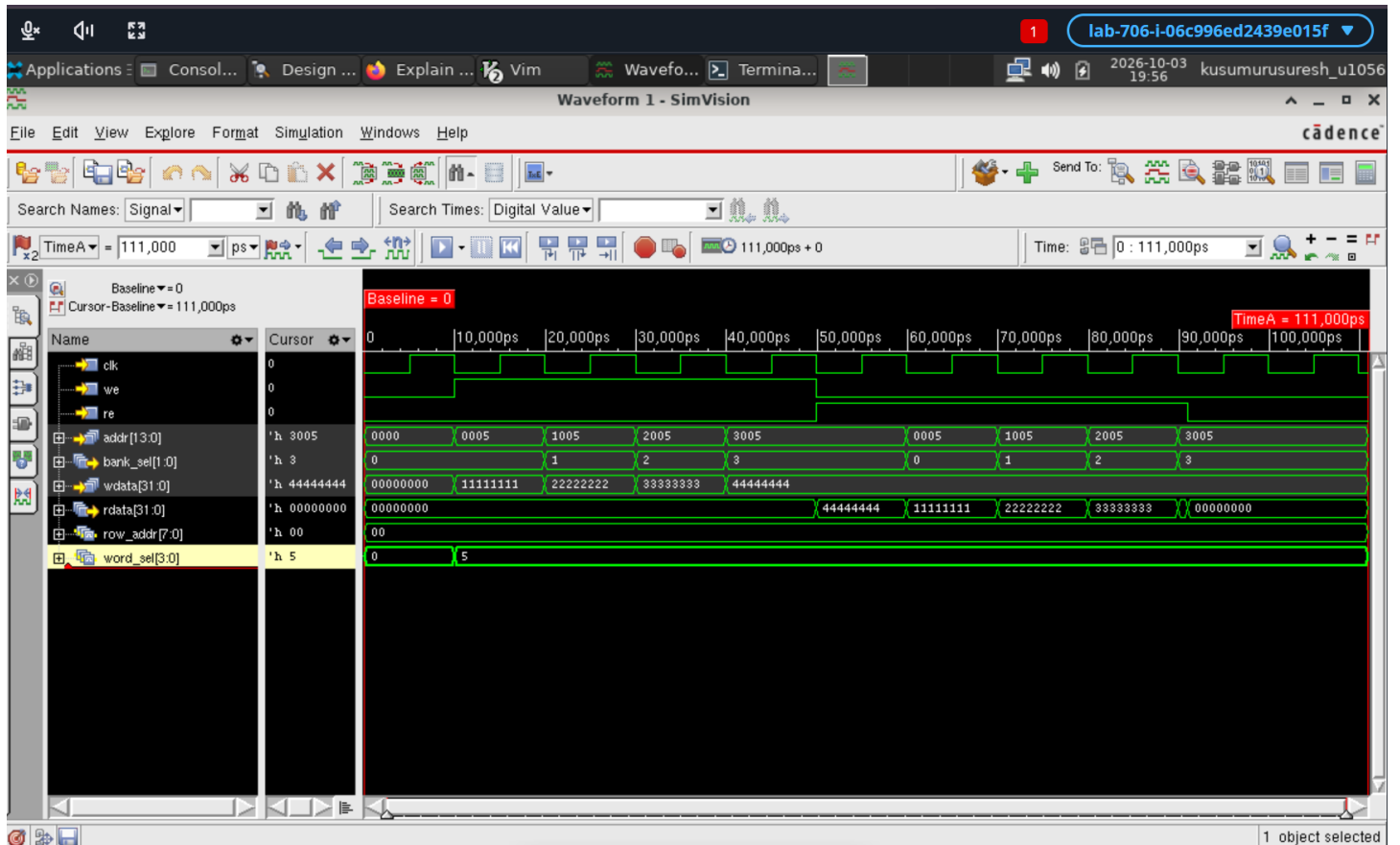
5. Cadence Xcelium simulation environment terminal:

```
will be used.
Loading snapshot worklib.tb_sram_4banks:sv ..... Done
xcelium> source /data/tools/DV/cdn/XCELIUM/tools/xcelium/files/xmsimrc
xcelium> run
xmsim: *W,DVEXACC: some objects excluded from $dumpvars due to access restrictions, use +access+r on command line for access to all objects.
File: ./tb_sram_4banks.sv, line = 49, pos = 16
Scope: tb_sram_4banks
Time: 0 FS + 0

=====
Q2(ii) FOUR-BANK SRAM WRITE
=====
WRITE | bank=0 | addr=0x0005 | row=0 | word=5 | data=0x11111111
WRITE | bank=1 | addr=0x1005 | row=0 | word=5 | data=0x22222222
WRITE | bank=2 | addr=0x2005 | row=0 | word=5 | data=0x33333333
WRITE | bank=3 | addr=0x3005 | row=0 | word=5 | data=0x44444444
=====
Q2(ii) FOUR-BANK SRAM READ
=====
READ | bank=0 | addr=0x0005 | expected=0x11111111 | actual=0x11111111 | PASS
READ | bank=1 | addr=0x1005 | expected=0x22222222 | actual=0x22222222 | PASS
READ | bank=2 | addr=0x2005 | expected=0x33333333 | actual=0x33333333 | PASS
READ | bank=3 | addr=0x3005 | expected=0x44444444 | actual=0x44444444 | PASS
=====
ASSIGNMENT 2(ii) TEST PASSED
ALL 4 BANKS VERIFIED
=====

Simulation complete via $finish(1) at time 111 NS + 0
./tb_sram_4banks.sv:192 $finish;
xcelium> exit
TOOL: xrun(64) 25.09-s009: Exiting on Oct 03, 2026 at 19:47:51 UTC (total: 00:00:04)
kusumurusuresh_u1056@lab-706-i-06c996ed2439e015f 4banks >
```

6. Cadence SimVision Waveform Viewer:





Assignment 3 — CNN Address Generation Across Staggered Banks:

1. cnn_input_sram.sv:

```
`timescale 1ns/1ps

module cnn_input_sram (
    input  wire      clk,

    input  wire      write_en,
    input  wire [1:0] write_bank,
    input  wire [13:0] write_addr,
    input  wire [7:0] write_data,

    input  wire      read_en,
    input  wire [7:0] start_row,
    input  wire [7:0] start_col,

    output reg [23:0] cnn_data,
    output reg      out_valid,

    output wire [1:0] bank0,
    output wire [1:0] bank1,
    output wire [1:0] bank2,

    output wire [13:0] addr0,
    output wire [13:0] addr1,
    output wire [13:0] addr2,

    output wire [7:0] pixel0,
    output wire [7:0] pixel1,
    output wire [7:0] pixel2
);

    // =====
    // 64 KB SRAM
    // 4 banks x 16 KB
    // =====

    reg [7:0] mem [0:65535];

    // =====
    // WRITE
    //
    // Physical address:
    // bank 0 -> 0x0000 - 0x3FFF
    // bank 1 -> 0x4000 - 0x7FFF
    // bank 2 -> 0x8000 - 0xBFFF
    // bank 3 -> 0xC000 - 0xFFFF
    // =====

    wire [15:0] write_phys_addr;

    assign write_phys_addr =
        {write_bank, write_addr};
```

```

always @(posedge clk) begin
    if (write_en)
        mem[write_phys_addr] <= write_data;
end

// =====
// THREE-PIXEL CNN WINDOW
//
// Pixel 0 = (start_row, start_col)
// Pixel 1 = (start_row, start_col+1)
// Pixel 2 = (start_row, start_col+2)
// =====

wire [8:0] col0;
wire [8:0] col1;
wire [8:0] col2;

assign col0 = {1'b0,start_col};
assign col1 = {1'b0,start_col} + 9'd1;
assign col2 = {1'b0,start_col} + 9'd2;

// =====
// STAGGERED BANK SELECTION
//
// column 0 -> bank 0
// column 1 -> bank 1
// column 2 -> bank 2
// column 3 -> bank 3
// column 4 -> bank 0
// ...
// =====

assign bank0 = col0[1:0];
assign bank1 = col1[1:0];
assign bank2 = col2[1:0];

// =====
// LOCAL ADDRESS INSIDE EACH BANK
//
// 256 rows x 64 bytes
//
// address = row*64 + column/4
// =====

assign addr0 =
    (start_row << 6) + col0[8:2];

assign addr1 =
    (start_row << 6) + col1[8:2];

assign addr2 =
    (start_row << 6) + col2[8:2];

// =====
// PHYSICAL ADDRESSES

```

```

// =====

wire [15:0] phys0;
wire [15:0] phys1;
wire [15:0] phys2;

assign phys0 = {bank0,addr0};
assign phys1 = {bank1,addr1};
assign phys2 = {bank2,addr2};

// =====
// READ PIXELS
// =====

assign pixel0 = mem[phys0];
assign pixel1 = mem[phys1];
assign pixel2 = mem[phys2];

// =====
// 24-BIT OUTPUT
// =====

always @(posedge clk) begin

    if (read_en) begin
        cnn_data <= {pixel0,pixel1,pixel2};
        out_valid <= 1'b1;
    end
    else begin
        cnn_data <= 24'h000000;
        out_valid <= 1'b0;
    end

end

endmodule

```

2.tb_cnn_input_sram.sv:

```

`timescale 1ns/1ps

// =====
// QUESTION 3 TESTBENCH
//
// Verifies:
// 1. Four-bank SRAM organization
// 2. Staggered address mapping
// 3. Bank selection
// 4. CNN read control
// 5. 24-bit output
// =====

module tb_cnn_input_sram;

    reg clk;

```



```

// Write interface
reg      write_en;
reg [1:0] write_bank;
reg [13:0] write_addr;
reg [7:0] write_data;

// CNN read interface
reg      read_en;
reg [7:0] start_row;
reg [7:0] start_col;

// Outputs
wire [23:0] cnn_data;
wire      out_valid;

wire [1:0] bank0;
wire [1:0] bank1;
wire [1:0] bank2;

wire [13:0] addr0;
wire [13:0] addr1;
wire [13:0] addr2;

wire [7:0] pixel0;
wire [7:0] pixel1;
wire [7:0] pixel2;

integer errors;

// =====
// DUT
// =====

cnn_input_sram dut (

    .clk(clk),

    .write_en(write_en),
    .write_bank(write_bank),
    .write_addr(write_addr),
    .write_data(write_data),

    .read_en(read_en),
    .start_row(start_row),
    .start_col(start_col),

    .cnn_data(cnn_data),
    .out_valid(out_valid),

    .bank0(bank0),
    .bank1(bank1),
    .bank2(bank2),

    .addr0(addr0),
    .addr1(addr1),

```

```

        .addr2(addr2),

        .pixel0(pixel0),
        .pixel1(pixel1),
        .pixel2(pixel2)
    );

    // =====
    // CLOCK
    // =====

    initial begin
        clk = 1'b0;

        forever #5 clk = ~clk;
    end

    // =====
    // WRITE TASK
    // =====

    task write_pixel;

        input [7:0] row;
        input [7:0] col;
        input [7:0] data;

        integer bank_number;
        integer local_col;
        integer memory_address;

        begin

            bank_number = col % 4;
            local_col = col / 4;
            memory_address = row * 64 + local_col;

            @(negedge clk);

            write_en    = 1'b1;
            write_bank  = bank_number;
            write_addr  = memory_address;
            write_data  = data;

            @(posedge clk);

            #1;

            write_en = 1'b0;

        end

    endtask

    // =====

```

```

// READ / VERIFY TASK
// =====

task read_window;

    input [7:0] row;
    input [7:0] col;

    input [7:0] expected0;
    input [7:0] expected1;
    input [7:0] expected2;

    reg [23:0] expected;

    begin

        expected = {expected0, expected1, expected2};

        @(negedge clk);

        start_row = row;
        start_col = col;
        read_en    = 1'b1;

        @(posedge clk);

        #1;

        $display("");
        $display("-----");
        $display("CNN READ");
        $display("Start Row      = %0d", row);
        $display("Start Column   = %0d", col);

        $display("Pixel 0      = 0x%02h", pixel0);
        $display("Pixel 1      = 0x%02h", pixel1);
        $display("Pixel 2      = 0x%02h", pixel2);

        $display("Bank selection = %0d, %0d, %0d",
                bank0, bank1, bank2);

        $display("Address      = %0d, %0d, %0d",
                addr0, addr1, addr2);

        $display("Expected output = 0x%06h", expected);
        $display("Actual output  = 0x%06h", cnn_data);

        if ((cnn_data === expected) &&
            (out_valid === 1'b1)) begin

            $display("RESULT      = PASS");

        end
    else begin

```

```

        $display("RESULT          = FAIL");
        errors = errors + 1;

    end

    read_en = 1'b0;

end

endtask

// =====
// TEST
// =====

initial begin

    errors = 0;

    write_en  = 1'b0;
    write_bank = 2'd0;
    write_addr = 14'd0;
    write_data = 8'h00;

    read_en   = 1'b0;
    start_row = 8'd0;
    start_col = 8'd0;

    // =====
    // VCD
    // =====

    $dumpfile("cnn_input_sram.vcd");
    $dumpvars(0, tb_cnn_input_sram);

    // =====
    // Header
    // =====

    $display("");
    $display("=====");
    $display("          QUESTION 3 - CNN INPUT SRAM");
    $display("=====");
    $display("Image      : 256 x 256");
    $display("Pixel      : 8 bits");
    $display("Banks      : 4");
    $display("Bank size  : 256 x 64 bytes");
    $display("Output     : 24 bits");
    $display("=====");

    // =====
    // WRITE KNOWN PIXELS
    //
    // Row 0:
    //

```

```

// col 0 = A0
// col 1 = A1
// col 2 = A2
// col 3 = A3
// col 4 = A4
// col 5 = A5
// col 6 = A6
// col 7 = A7
// col 8 = A8
// col 9 = A9
// =====

write_pixel(8'd0, 8'd0, 8'hA0);
write_pixel(8'd0, 8'd1, 8'hA1);
write_pixel(8'd0, 8'd2, 8'hA2);
write_pixel(8'd0, 8'd3, 8'hA3);
write_pixel(8'd0, 8'd4, 8'hA4);
write_pixel(8'd0, 8'd5, 8'hA5);
write_pixel(8'd0, 8'd6, 8'hA6);
write_pixel(8'd0, 8'd7, 8'hA7);
write_pixel(8'd0, 8'd8, 8'hA8);
write_pixel(8'd0, 8'd9, 8'hA9);

// =====
// Row 1
// =====

write_pixel(8'd1, 8'd0, 8'hB0);
write_pixel(8'd1, 8'd1, 8'hB1);
write_pixel(8'd1, 8'd2, 8'hB2);
write_pixel(8'd1, 8'd3, 8'hB3);
write_pixel(8'd1, 8'd4, 8'hB4);
write_pixel(8'd1, 8'd5, 8'hB5);

// =====
// READ WINDOWS
// =====

// -----
// Window 1: columns 0,1,2
// Banks = 0,1,2
// -----

read_window(
    8'd0,
    8'd0,
    8'hA0,
    8'hA1,
    8'hA2
);

// -----
// Window 2: columns 1,2,3
// Banks = 1,2,3
// -----

```

```

read_window(
    8'd0,
    8'd1,
    8'hA1,
    8'hA2,
    8'hA3
);

// -----
// Window 3: columns 2,3,4
// Banks = 2,3,0
// -----

read_window(
    8'd0,
    8'd2,
    8'hA2,
    8'hA3,
    8'hA4
);

// -----
// Window 4: columns 3,4,5
// Banks = 3,0,1
// -----

read_window(
    8'd0,
    8'd3,
    8'hA3,
    8'hA4,
    8'hA5
);

// -----
// Window 5: columns 7,8,9
// Banks = 3,0,1
// -----

read_window(
    8'd0,
    8'd7,
    8'hA7,
    8'hA8,
    8'hA9
);

// -----
// Window 6: second row
// -----

read_window(
    8'd1,
    8'd0,

```

```

        8'hB0,
        8'hB1,
        8'hB2
    );

    // =====
    // FINAL RESULT
    // =====

    $display("");
    $display("=====");

    if (errors == 0) begin

        $display("        QUESTION 3 TEST PASSED");
        $display("        CNN ADDRESS GENERATION : PASS");
        $display("        BANK SELECTION          : PASS");
        $display("        READ CONTROL            : PASS");
        $display("        24-BIT OUTPUT           : PASS");

    end
    else begin

        $display("        QUESTION 3 TEST FAILED");
        $display("        ERRORS = %0d", errors);

    end

    $display("=====");
    $display("");

    #20;

    $finish;

end

endmodule

```

3. terminal:

The screenshot shows a VS Code terminal window with the following content:

```
File Edit Selection View Go ... 3 [WSL: Ubuntu]

Explorer:
- Open Editors
- Welcome
- cnm_input_sram.v
- tb_cnm_input_sram.v
- 3 [WSL: Ubuntu]
  - cnm_input_sram_sim
  - cnm_input_sram.vcd
  - tb_cnm_input_sram.v

Terminal:
sureshvis@DESKTOP-CB90UTN:~/IITKGP_assignment1/3$ rm -f cnm_input_sram_sim cnm_input_sram.vcd
sureshvis@DESKTOP-CB90UTN:~/IITKGP_assignment1/3$ iverilog -g2012 -o cnm_input_sram_sim cnm_input_sram.v tb_cnm_input_sram.v
VCD info: dumpFile cnm_input_sram.vcd opened for output.

=====
QUESTION 3 - CNM INPUT SRAM
=====
Image      : 256 x 256
Pixel 0    : 8 bits
Banks      : 4
Bank size  : 256 x 64 bytes
Output     : 24 bits
=====

CNM READ
Start Row  = 0
Start Column = 0
Pixel 0    = 0xa0
Pixel 1    = 0xa1
Pixel 2    = 0xa2
Bank selection = 0, 0, 0
Address    = 0xa0a1a2
Expected output = 0xa0a1a2
Actual output = 0xa0a1a2
RESULT     = PASS

CNM READ
Start Row  = 0
Start Column = 1
Pixel 0    = 0xa1
Pixel 1    = 0xa2
Pixel 2    = 0xa3
Bank selection = 1, 0, 0
Address    = 0, 0, 0
Expected output = 0xa1a2a3
Actual output = 0xa1a2a3
RESULT     = PASS

CNM READ
Start Row  = 0
Start Column = 2
Pixel 0    = 0xa2
Pixel 1    = 0xa3
Pixel 2    = 0xa4
Bank selection = 2, 0, 0
Address    = 0, 0, 0
Expected output = 0xa2a3a4
Actual output = 0xa2a3a4
RESULT     = PASS

CNM READ
Start Row  = 0
Start Column = 3
Pixel 0    = 0xa3
Pixel 1    = 0xa4
Pixel 2    = 0xa5
Bank selection = 3, 0, 0
Address    = 0, 0, 0
Expected output = 0xa3a4a5
Actual output = 0xa3a4a5
RESULT     = PASS

Bank selection = 2, 3, 0
Address       = 0, 0, 1
Expected output = 0xa2a3a4
Actual output = 0xa2a3a4
RESULT       = PASS

sureshvis@DESKTOP-CB90UTN:~/IITKGP_assignment1/3$ vvp cnm_input_sram_sim

CNM READ
Start Row  = 0
Start Column = 3
Pixel 0    = 0xa3
Pixel 1    = 0xa4
Pixel 2    = 0xa5
Bank selection = 3, 0, 1
Address    = 0, 1, 1
Expected output = 0xa3a4a5
Actual output = 0xa3a4a5
RESULT     = PASS

CNM READ
Start Row  = 0
Start Column = 7
Pixel 0    = 0xa7
Pixel 1    = 0xa8
Pixel 2    = 0xa9
Bank selection = 3, 0, 1
Address    = 1, 1, 2
Expected output = 0xa7a8a9
Actual output = 0xa7a8a9
RESULT     = PASS

CNM READ
Start Row  = 1
Start Column = 0
Pixel 0    = 0xb0
Pixel 1    = 0xb1
Pixel 2    = 0xb2
Bank selection = 0, 1, 2
Address    = 64, 64, 64
Expected output = 0xb0b1b2
Actual output = 0xb0b1b2
RESULT     = PASS

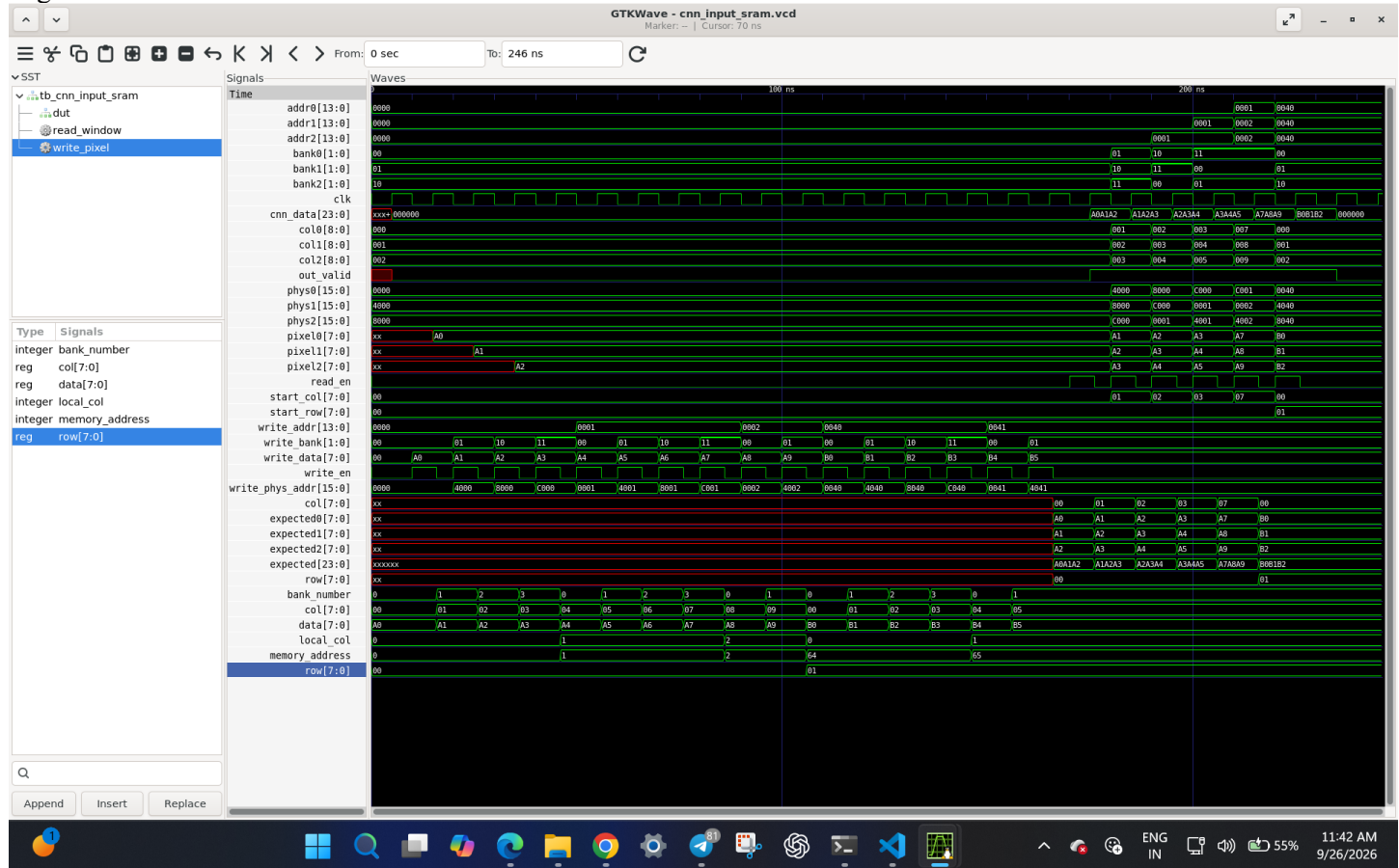
=====
QUESTION 3 TEST PASSED
CNM ADDRESS GENERATION : PASS
BANK SELECTION        : PASS
READ CONTROL          : PASS
24-BIT OUTPUT         : PASS
=====

tb_cnm_input_sram.v:381: $finish called at 240800 (1ps)
sureshvis@DESKTOP-CB90UTN:~/IITKGP_assignment1/3$ gtime cnm_input_sram.vcd &
[1] 75130
sureshvis@DESKTOP-CB90UTN:~/IITKGP_assignment1/3$
GTime Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[240800] end time.
sureshvis@DESKTOP-CB90UTN:~/IITKGP_assignment1/3$
```

The terminal output shows the execution of a Verilog testbench for a 32-bit SRAM. It includes a memory dump, several read operations, and test results. The test results show that the CNM ADDRESS GENERATION, BANK SELECTION, READ CONTROL, and 24-BIT OUTPUT tests all passed. The testbench was executed using the 'vvp' command, and the results were saved to a VCD file named 'cnm_input_sram.vcd'.

4. gtkwave:



5. Cadence Xcelium simulation environment terminal:

```

lab-706-i-06c996ed2439e015f
Applications Analyse SRAM CNN M... Vim Terminal - 2026-10-03 20:05 kusumurusuresh_u1056
Terminal -
File Edit View Terminal Tabs Help
Start Column = 7
Pixel 0 = 0xa7
Pixel 1 = 0xa8
Pixel 2 = 0xa9
Bank selection = 3, 0, 1
Address = 1, 2, 2
Expected output = 0xa7a8a9
Actual output = 0xa7a8a9
RESULT = PASS

-----
CNN READ
Start Row = 1
Start Column = 0
Pixel 0 = 0xb0
Pixel 1 = 0xb1
Pixel 2 = 0xb2
Bank selection = 0, 1, 2
Address = 64, 64, 64
Expected output = 0xb0b1b2
Actual output = 0xb0b1b2
RESULT = PASS

=====
QUESTION 3 TEST PASSED
CNN ADDRESS GENERATION : PASS
BANK SELECTION : PASS
READ CONTROL : PASS
24-BIT OUTPUT : PASS
=====

Simulation complete via $finish(1) at time 246 NS + 0
./tb_cnn_input_sram.sv:381 $finish;
xcelium> exit
TOOL: xrun(64) 25.09-s009: Exiting on Oct 03, 2026 at 20:05:33 UTC (total: 00:00:04)
kusumurusuresh_u1056@lab-706-i-06c996ed2439e015f 3 >

```

6. Cadence SimVision Waveform Viewer:

